

GitHub - Part 3

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

This study guide, "GitHub (Part 3/3): Production SRE, Diagnostics, Troubleshooting, Custom Alerting, and Incident Response," is meticulously crafted for professionals with 6+ years of IT experience aiming to become industry experts in GitHub operations. It delves into the deepest layers of GitHub Enterprise Server (GHES) and its ecosystem from a rigorous Site Reliability Engineering (SRE) perspective, providing insights crucial for maintaining highly available, performant, and secure development platforms in a production environment.

1. Part Introduction and Scope

This final part of our GitHub study guide shifts focus from usage and automation to the critical domain of Site Reliability Engineering (SRE) and Operations for GitHub, particularly GitHub Enterprise Server (GHES). While GitHub.com provides a managed service, large enterprises often opt for GHES to meet stringent compliance, security, and network isolation requirements. Managing GHES at scale demands a deep understanding of its internal architecture, replication mechanisms, monitoring capabilities, and disaster recovery procedures.

Our scope will cover:

- **Production Readiness:** Ensuring GHES instances are robust, scalable, and secure for critical development workloads.
- **Observability:** Leveraging built-in metrics, logs, and external monitoring tools (like Prometheus) for comprehensive health visibility.
- **Diagnostics & Troubleshooting:** Methodologies and tools to pinpoint and resolve issues rapidly, minimizing Mean Time To Recovery (MTTR).
- **Alerting & Incident Response:** Crafting effective alerting rules and developing structured runbooks for various incident scenarios.
- **Root Cause Analysis (RCA):** Techniques for deep-diving into past incidents to prevent recurrence.
- **Security & Hardening:** Advanced practices to protect the GHES environment and integrated components like self-hosted runners.

This section assumes a foundational understanding of GitHub's core features, CI/CD, and basic cloud infrastructure concepts, as covered in Part 1 and Part 2.

2. Why this part's concepts are critical for high-availability systems

GitHub, especially GHES, is often the single source of truth for an organization's intellectual property (source code), the trigger for all automated deployments, and a central hub for collaborative development. Its availability and performance directly impact:

- **Developer Productivity:** Downtime or slow performance means developers cannot commit code, review pull requests, or trigger builds, halting critical workstreams.
- **Deployment Velocity:** CI/CD pipelines often start with a Git push or a pull request merge. An

unavailable GitHub instance directly prevents new deployments, impacting feature delivery and bug fixes.

- **Incident Response:** During an ongoing incident, access to source code, runbooks stored in repositories, and the ability to deploy hotfixes are paramount. If GitHub itself is down, the ability to respond effectively is severely crippled.
- **Security & Compliance:** GHES hosts sensitive data. Operational practices ensure data integrity, prevent unauthorized access, and help meet regulatory requirements through robust auditing and secure configurations.
- **Business Continuity:** In the event of a regional outage or disaster, GHES must be recoverable quickly to restore core development and operational capabilities, demonstrating resilience and minimizing business impact.

Mastering these SRE and operational concepts transforms GitHub from a mere code repository into a resilient, highly available, and performant platform that underpins the entire software delivery lifecycle, critical for any modern, high-velocity enterprise.

3. Real-world Enterprise Use Cases with architecture-level details

Use Case 1: High Availability (HA) and Disaster Recovery (DR) for GitHub Enterprise Server

Scenario: A large financial institution requires continuous availability of its GHES instance, with automatic failover in case of an instance failure and a robust DR strategy for regional outages.

Architecture Details:

- **Primary Instance:** Deployed in `Region A, Availability Zone 1` (e.g., AWS EC2 `m5.xlarge` with EBS `gp3` for storage). All read/write operations occur here.
- **Replica Instance (HA):** Deployed in `Region A, Availability Zone 2` (same instance type/storage). This replica maintains a real-time, block-level synchronization with the primary via `ghe-repl-setup` using `rsync` for Git data, `PostgreSQL` streaming replication for database, and `Elasticsearch` for search indexes. Network latency between AZs is low (<5ms).
- **Load Balancer (L4/L7):** An internal Load Balancer (e.g., AWS NLB or ALB) fronts both primary and replica instances. DNS records (`github.company.com`) point to the LB CNAME. The LB health checks monitor GHES application health endpoints (e.g., `/status`, `/healthz`). In normal operation, it directs traffic to the primary. Upon primary failure, manual or automated promotion of the replica is triggered.
- **Geo-replica Instance (DR):** Deployed in `Region B, Availability Zone 1` (e.g., a completely separate AWS region). This replica is configured for asynchronous replication from the primary in Region A. This provides protection against a full regional outage in Region A. Replication lag is expected to be higher due to inter-region latency.
- **Storage:** EBS volumes are used for the primary, HA replica, and DR replica. Snapshots are taken regularly for point-in-time recovery. Git LFS objects are typically stored on S3-compatible object storage, which offers high durability and geo-replication capabilities.

- **External Services:**
- **Identity Provider:** SAML/LDAP integration for user authentication (e.g., Okta, Active Directory Federation Services).
- **SMTP:** For email notifications (e.g., Amazon SES, SendGrid).
- **DNS:** Route 53 or internal DNS for `github.company.com` resolution.
- **Monitoring:** Centralized Prometheus/Grafana stack scrapes metrics from all GHES instances; Loki/Splunk for log aggregation.
- **Network:** All GHES instances are in private subnets, accessible only through a bastion host or VPN/Direct Connect from internal networks. Security Groups/Network ACLs tightly control inbound/outbound traffic (e.g., port 8443 for web UI, 122 for SSH Git, 8080/8443 for internal replication).

Use Case 2: Secure and Scalable Self-Hosted GitHub Actions Runners

Scenario: A software company needs to execute sensitive build jobs on specific hardware configurations (e.g., GPU machines, custom toolchains) within its private network, demanding dynamic scaling and robust security.

Architecture Details:

- **GitHub Enterprise Server:** The central control plane for Actions workflows, hosted as per Use Case 1.
- **Runner Orchestration:**
- **Cloud Provider (e.g., AWS Auto Scaling Group, Azure VMSS, GCP MIG):** Runners are deployed as ephemeral instances.
- **Kubernetes Cluster:** Runners are deployed as pods within a dedicated namespace, leveraging Kubernetes' auto-scaling (HPA, Cluster Autoscaler) and resource management. Each job gets a fresh pod.
- **Runner Image:** A hardened AMI/Docker image containing the `actions-runner` agent, necessary build tools, and security agents. This image is regularly updated and scanned.
- **Network Isolation:**
- Runners operate in private subnets, with egress controlled by Network ACLs and Security Groups.
- Access to internal resources (e.g., artifact repositories, databases) is strictly whitelisted.
- In Kubernetes, Network Policies restrict pod-to-pod communication.
- **Secret Management:**
- GitHub Actions secrets are encrypted at rest.
- For runners requiring access to external secrets (e.g., cloud provider credentials), integration with a centralized secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault, Azure Key Vault) is implemented. Runners fetch secrets at runtime using short-lived credentials (e.g., IAM roles, K8s Service Account tokens).
- **Monitoring:** Runner health (online/offline status, job execution time, resource utilization) is monitored via GHES metrics, cloud provider metrics, and dedicated agents (e.g., Prometheus `node\_exporter` on VMs, `kube-state-metrics` on Kubernetes).
- **Ephemeral Nature:** Runners are configured to be ephemeral. After each job, the runner instance/pod is terminated and a new one is provisioned for the next job, preventing state contamination and reducing attack surface.

- **Pre-receive Hooks:** Custom pre-receive hooks on GHES enforce policies like ensuring all commits are signed, or blocking commits containing sensitive patterns, before they even hit the repository.

4. Comprehensive Architecture Explanation

This section focuses on the GitHub Enterprise Server (GHES) architecture, as it's the component directly managed and operated by SRE teams in an enterprise context.

Textual Explanation:

GitHub Enterprise Server is a self-hosted platform for Git repository management and software development, designed for enterprise-grade security, compliance, and scalability. Its architecture is distributed, consisting of several core services that work in concert.

1. Application Tier: This is the primary VM or appliance where the GitHub application runs. It encapsulates most of the core services.

- **Git Storage (Filesystem):** Houses all Git repositories. This is typically a large, high-performance block storage volume (e.g., EBS, Azure Disk, GCP Persistent Disk) mounted to the GHES instance. Git LFS (Large File Storage) objects are often offloaded to external S3-compatible object storage.
- **Database (PostgreSQL):** Stores all GitHub metadata, including user accounts, repository information, pull request data, issues, comments, and configuration settings.
- **Search (Elasticsearch):** Powers the code search and issue/pull request search functionality. It indexes the Git repositories and database content for fast retrieval.
- **Web UI & API (Ruby on Rails, Go, etc.):** The user interface and the programmatic API endpoints for interaction with GitHub.
- **Job Queues (Sidekiq/Redis):** Handles background tasks such as webhook deliveries, email notifications, repository indexing, and internal maintenance operations. Redis typically acts as the broker for these queues.
- **Redis Cache:** Used for various caching mechanisms to improve performance, such as session data and frequently accessed configuration.
- **Git Protocol Services (SSH, HTTPS):** Handles all Git operations (clone, push, pull) over SSH (port 122) and HTTPS (port 8443).
- **Internal Services:** Various other services manage logging, metrics exposition, system configuration, and internal communication.

2. High Availability (HA) Configuration:

- GHES supports an active/passive HA configuration. A **Primary Instance** handles all write operations and serves reads. A **Replica Instance** asynchronously replicates data from the primary.
- Replication mechanisms include:
 - **Git Replication:** `rsync` or custom block-level sync for the Git filesystem.
 - **Database Replication:** PostgreSQL streaming replication.

- **Search Replication:** Elasticsearch snapshot/restore or direct indexing.
- A **Load Balancer** (L4/L7) typically sits in front of the primary and replica, routing traffic to the active primary and facilitating failover.

3. Disaster Recovery (DR) / Geo-replication:

- A **Geo-replica Instance** can be configured in a geographically separate region, providing protection against regional outages. This is also an asynchronous replication but with potentially higher latency.
- **Backup Solutions:** `ghe-backup` or `ghe-migrator` create logical backups that can be restored to a new GHES instance. These backups often target S3-compatible storage.

4. External Integrations:

- **Identity Provider:** SAML or LDAP for user authentication and authorization.
- **SMTP Server:** For email notifications.
- **DNS:** For resolving the GHES hostname.
- **Monitoring & Logging:** External systems like Prometheus/Grafana, Splunk, ELK stack collect metrics and logs from GHES instances.
- **Object Storage:** For Git LFS, repository archives, and backups.

5. GitHub Actions Self-hosted Runners:

- Separate compute instances (VMs, containers) registered with GHES to execute CI/CD workflows. They communicate with GHES over HTTPS and are typically managed by auto-scaling groups or Kubernetes for elasticity and isolation.

Mermaid Diagram: GitHub Enterprise Server (GHES) Production Architecture

```
graph TD
    subgraph User_Access [User Access]
        User[User(Developers/Admins)] --> LB[LB(Load Balancer)];
        LB -->|HTTPS/SSH| GHES_Primary[GHES Primary Instance];
        LB -->|HTTPS/SSH| GHES_Replica[GHES HA Replica];
    end

    subgraph GHES_Primary_Instance [GHES Primary Instance]
        GHES_Primary --> GitStorage_P[Git Filesystem Storage: Block Store];
        GHES_Primary --> DB_P[DB(PostgreSQL DB)];
        GHES_Primary --> ES_P[Elasticsearch];
        GHES_Primary --> Redis_P[Redis Cache/Queues];
        GHES_Primary --> ObjectStorage[Object Storage: S3 for LFS/Backups];
    end

    subgraph GHES_HA_Replica [GHES HA Replica]
        GHES_Replica --> GitStorage_R[Git Filesystem Storage: Block Store];
        GHES_Replica --> DB_R[DB(PostgreSQL DB)];
        GHES_Replica --> ES_R[Elasticsearch];
        GHES_Replica --> Redis_R[Redis Cache/Queues];
    end
```

```

subgraph GHES_Geo-Replica (DR)
    GHES_GeoReplica(GHES Geo-Replica Instance) --> GitStorage_G(Git Filesystem Storage);
    GHES_GeoReplica --> DB_G(PostgreSQL DB);
    GHES_GeoReplica --> ES_G(Elasticsearch);
end

GHES_Primary --replication (sync)--> GHES_Replica;
GHES_Primary --replication (async)--> GHES_GeoReplica;

subgraph External_Services
    GHES_Primary --> IDP(SAML/LDAP Identity Provider);
    GHES_Primary --> SMTP(SMTP Server);
    GHES_Primary --> DNS(DNS Service);
end

subgraph Observability & Operations
    GHES_Primary --metrics--> Prometheus(Prometheus);
    GHES_Replica --metrics--> Prometheus;
    GHES_GeoReplica --metrics--> Prometheus;
    Prometheus --> Grafana(Grafana Dashboards);
    Prometheus --> Alertmanager(Alertmanager);

    GHES_Primary --logs--> LogAgg(Log Aggregation: ELK/Splunk);
    GHES_Replica --logs--> LogAgg;
    GHES_GeoReplica --logs--> LogAgg;

    GHES_Primary --audit logs--> SIEM(SIEM);
end

subgraph GitHub_Actions_Runners
    GHES_Primary --job assignments--> RunnerOrch(Runner Orchestration: Auto Scaling
Group/K8s);
    RunnerOrch --> SelfHostedRunners(Ephemeral Self-Hosted Runners);
    SelfHostedRunners --> SecretsMgr(Secrets Manager);
    SelfHostedRunners --> ArtifactRepo(Artifact Repository);
end

style GHES_Primary fill:#f9f,stroke:#333,stroke-width:2px;
style GHES_Replica fill:#ccf,stroke:#333,stroke-width:2px;
style GHES_GeoReplica fill:#cfc,stroke:#333,stroke-width:2px;
style LB fill:#fff,stroke:#333,stroke-width:2px;

```

5. Types, classifications, or components relating to this part's focus

When focusing on SRE, diagnostics, and troubleshooting for GitHub, we consider both the managed GitHub.com services (from a consumer's operational perspective) and the self-managed GitHub Enterprise Server (GHES).

I. GitHub Enterprise Server (GHES) Components (Internal SRE Focus)

1. Core Services:

- **Git (Git Daemon, Git Smart HTTP):** Handles all repository operations (clone, push, pull).
- **Database (PostgreSQL):** Stores all relational data. Critical for user management, repository metadata, PRs, issues.
- **Search (Elasticsearch):** Indexes code and content for search functionality.
- **Web Application (Ruby on Rails/Go):** Serves the UI and API requests.
- **Background Jobs (Sidekiq/Redis):** Asynchronously processes tasks like webhooks, notifications, repository cleanup.
- **Redis Cache:** In-memory data store for caching and queue management.
- **Filesystem (GFS/LFS):** Stores Git repository data and large files (Git LFS).
- **Monitoring Agent (collectd, Prometheus exporter):** Collects system and application metrics.

2. Infrastructure & Operational Components:

- **Appliance VM:** The underlying virtual machine (e.g., EC2, Azure VM) hosting the GHES software.
- **Storage:** Block storage (e.g., EBS, Azure Disk) for Git data and databases; Object storage (S3-compatible) for LFS and backups.
- **Load Balancer:** Distributes traffic, provides health checks, and enables HA failover.
- **DNS:** Resolves the GHES hostname.
- **Network (VPC/VNet, Subnets, Security Groups/ACLs):** Controls network access and segmentation.
- **Replication Engine (`ghe-repl`):** Manages data synchronization between primary and replica instances.
- **Backup & Restore (`ghe-backup`, `ghe-restore`):** Tools for data protection and recovery.
- **Management Console:** Web UI for configuring GHES system settings.
- **SSH Console (`ghe-ssh`):** Admin access for troubleshooting and advanced configuration.

3. Related/Integrated Services:

- **GitHub Actions Self-Hosted Runners:** External compute resources executing CI/CD workflows.
- **Identity Provider (SAML/LDAP):** For user authentication.
- **SMTP Server:** For email notifications.
- **External Monitoring Stack (Prometheus, Grafana, Alertmanager):** For comprehensive observability.
- **Centralized Logging (Splunk, ELK, Loki):** For aggregating GHES system and application logs.

II. GitHub.com Services (Consumer SRE Focus)

While you don't manage GitHub.com's infrastructure, your SRE role involves ensuring your applications and workflows reliably interact with it.

1. GitHub APIs (REST/GraphQL):

- **Rate Limits:** Understanding and managing API rate limits to prevent throttling.
- **Availability/Latency:** Monitoring GitHub's status page, and implementing retry logic with exponential backoff.
- **Authentication:** Personal Access Tokens (PATs), GitHub Apps, OAuth Apps.

2. Webhooks:

- **Delivery Reliability:** Monitoring webhook delivery status, handling failures, implementing idempotency in receivers.
- **Secrets:** Securely handling webhook secrets for payload verification.
- **Event Types:** Understanding the various events and their payloads.

3. GitHub Actions (Cloud-hosted Runners):

- **Service Availability:** Relying on GitHub's operational status.
- **Resource Limits:** Understanding concurrency limits, artifact storage limits.
- **Marketplace Actions:** Ensuring security and reliability of third-party actions.

4. Codespaces:

- **Resource Management:** Understanding billing and resource consumption if using Codespaces.
- **Customization:** Managing custom dev container configurations.

This classification helps delineate responsibilities and focus SRE efforts effectively - direct infrastructure management for GHES, and robust integration design for GitHub.com services.

6. Step-by-step production implementation guide

This guide focuses on setting up a highly available GHES instance with robust monitoring and backup.

Step 1: Initial GHES Primary Instance Deployment

1. Provision Base VM: Select an appropriate cloud instance type (e.g., AWS EC2 `m5.xlarge` or larger) with sufficient CPU, RAM, and network bandwidth. Attach a high-performance block storage volume (e.g., 500GB+ `gp3` SSD) for the `/data` partition.
 - **Security:** Place the VM in a private subnet. Configure Security Groups/Network ACLs to allow inbound SSH (port 22) from a bastion host, HTTPS (port 8443) from the Load Balancer, and internal replication ports (e.g., 8080, 122) from the replica.
2. Install GHES: Download the GHES appliance image for your cloud provider (AMI, VHD, OVA). Launch the VM using this image.
3. Initial Configuration:
 - Access the GHES Management Console via HTTPS (e.g., `https://<GHES\_IP>:8443`).

- Upload your license file.
 - Set the root password.
 - Configure basic network settings, DNS, NTP.
 - **Crucially, enable SSH access for `ghe-admin` user.**
 - Choose "Standalone" mode initially.
4. DNS Setup: Create a CNAME or A record (e.g., `github.company.com`) pointing to the GHES instance's private IP or the Load Balancer IP (if already set up).
 5. SSL/TLS: Upload your enterprise's wildcard SSL certificate (e.g., `\*.company.com`) and private key to the GHES Management Console. Ensure the certificate chain is complete.
 6. Reboot & Verify: Save settings, reboot the instance, and verify GHES is accessible via the configured hostname and HTTPS.

Step 2: Configure High Availability (HA) Replica

1. Provision Replica VM: Launch another GHES appliance VM in a different Availability Zone within the same region, identical to the primary in instance type and storage. Ensure network connectivity between primary and replica.
2. Initial Replica Setup: Access its Management Console (`https://<REPLICA\_IP>:8443`). Upload license, set root password. For now, select "Standalone" mode.
3. Establish Replication Link:

- On the **primary** GHES instance, generate a replication key:

```
ghe-repl-setup -p $(cat ~/.ssh/id_rsa.pub)
```

This command outputs a long public key. Copy this key.

- On the **replica** GHES instance, add the primary's public key and initiate replication. Replace `` with the primary's private IP:

```
ghe-repl-setup <PRIMARY_IP> -i <PRIMARY_PUBLIC_KEY_COPIED_ABOVE>
```

- Confirm the replication setup from the primary:

```
ghe-repl-status
```

Wait for `OK` status and `Last synced` time to update.

4. Load Balancer Integration:

- Configure an internal Load Balancer (e.g., AWS NLB).
- Add both the primary and replica GHES instances as targets.
- Configure health checks against a GHES health endpoint (e.g., TCP 8443 or `https://github.company.com/status`).
- Ensure the LB is configured to direct traffic *only* to the primary instance initially. This often involves weighting or active/passive configurations, or simply having the replica instance marked as unhealthy in the LB until a failover is required.

- Update DNS `github.company.com` to point to the Load Balancer's CNAME or IP.

Step 3: Implement Automated Backups

1. Provision Backup Host: Create a separate lightweight VM (e.g., `t3.medium`) in a secure private subnet.
2. Install `ghe-backup`:

```
curl -L
https://github.com/github/backup-utils/releases/download/<LATEST_VERSION>/github-backup-utils
.tgz | tar xz -C /usr/local/bin --strip-components=1
```

3. Configure Backup: Create `~/.ssh/id\_rsa` on the backup host and add its public key to the GHES primary's authorized_keys (via Management Console or `ghe-ssh`).

Create a backup configuration file (e.g., `/etc/ghe-backup.config`):

```
# /etc/ghe-backup.config
GHE_HOSTS="github.company.com" # Or primary IP
GHE_DATA_DIR="/mnt/github-backups"
S3_BUCKET="s3://my-github-backup-bucket/daily"
S3_ENDPOINT="https://s3.<region>.amazonaws.com" # Or other S3 compatible
GHE_SSH_USER="admin" # Or 'ghe-admin' if using older setup
GHE_SSH_PORT=122 # Or 22 if you changed it
GHE_KEEP_DAILY_BACKUPS=7
GHE_KEEP_WEEKLY_BACKUPS=4
GHE_KEEP_MONTHLY_BACKUPS=6
```

4. Create Backup Script & Cron:

```
#!/bin/bash
source /etc/ghe-backup.config
ghe-backup -v
```

Set up a cron job (e.g., `0 2 \* \* \* /usr/local/bin/ghe-backup-script.sh > /var/log/ghe-backup.log 2>&1`) to run nightly during off-peak hours.

5. Verify Backup: Confirm backup files appear in the S3 bucket. Perform a test restore to a throwaway GHES instance occasionally.

Step 4: Configure Observability (Prometheus & Log Aggregation)

1. Enable GHES Metrics: In the GHES Management Console, navigate to Monitoring and enable Prometheus metrics endpoint. Note the authentication token.
2. Prometheus Scrape Config: In your Prometheus configuration (`prometheus.yml`):

```
# prometheus.yml
scrape_configs:
  - job_name: 'github_enterprise_primary'
    scheme: https
    tls_config:
```

```

    insecure_skip_verify: true # Use proper CA bundle in production
  basic_auth:
    username: "admin" # Or specific monitoring user if configured
    password: "<GHES_MONITORING_TOKEN>" # From Management Console
  static_configs:
    - targets: ['<PRIMARY_GHES_IP>:8443']
  metrics_path: '/api/v3/enterprise/metrics'
  relabel_configs:
    - source_labels: [__address__]
      regex: '([^.:]+):.*'
      target_label: instance_ip
    - target_label: gh_instance_type
      replacement: 'primary'

- job_name: 'github_enterprise_replica'
  scheme: https
  tls_config:
    insecure_skip_verify: true
  basic_auth:
    username: "admin"
    password: "<GHES_MONITORING_TOKEN>"
  static_configs:
    - targets: ['<REPLICA_GHES_IP>:8443']
  metrics_path: '/api/v3/enterprise/metrics'
  relabel_configs:
    - source_labels: [__address__]
      regex: '([^.:]+):.*'
      target_label: instance_ip
    - target_label: gh_instance_type
      replacement: 'replica'

# Add node_exporter for host-level metrics if installed
- job_name: 'node_exporter_ghes_primary'
  static_configs:
    - targets: ['<PRIMARY_GHES_IP>:9100']
- job_name: 'node_exporter_ghes_replica'
  static_configs:
    - targets: ['<REPLICA_GHES_IP>:9100']

```

Install `node\_exporter` on GHES instances for OS-level metrics.

3. Grafana Dashboards: Import or create Grafana dashboards for GHES. GitHub provides official examples, or create custom ones using key metrics like `github\_replication\_status\_health`, `node\_cpu\_seconds\_total`, `node\_filesystem\_avail\_bytes`, `github\_http\_requests\_total`, `github\_job\_queues\_length`.

4. Log Aggregation: Configure GHES to send logs via `syslog-ng` to your centralized log aggregation system (e.g., Splunk, ELK, Loki). This is configured in the Management Console under "Monitoring -> Log Forwarding".

7. Standard CLI commands with deep technical explanations of each flag

For GitHub Enterprise Server (GHES), many critical SRE operations are performed via SSH using `ghe-` utilities.

1. `ghe-repl-setup` - Sets up or modifies a replica instance.

```
ghe-repl-setup <primary_ip> -i <public_key> [-v --wait --config-url <URL> --admin-token <TOKEN>]
```

- ``<primary_ip>``: The IP address of the primary GHES instance.
- ``-i <public_key>``: ****Crucial.**** Specifies the SSH public key of the primary instance's `ghe-admin` user. This key is used by the replica to authenticate and establish the initial SSH tunnel for replication. On the primary, you'd typically generate this with `ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa_ghe_repl` and then use `cat ~/.ssh/id_rsa_ghe_repl.pub` to get the public key. This `ghe-repl-setup` command on the *replica* will then use this key to connect to the *primary*.
- ``-v``: Enables verbose output, showing detailed progress of the replication setup. Useful for debugging.
- ``--wait``: The command will wait until the initial synchronization of the replica is complete before exiting. Essential for scripting automated HA setups.
- ``--config-url <URL>``: Specifies an alternative URL for the primary's Management Console. Useful if the primary is behind a load balancer with a different management interface.
- ``--admin-token <TOKEN>``: Provides the Management Console admin token for authentication, avoiding interactive password prompts. Useful for automation.

2. `ghe-repl-status` - Displays the current replication status.

```
ghe-repl-status [-v --json]
```

- ``-v``: Verbose output, showing detailed status for each component (Git, MySQL, Elasticsearch, Redis, etc.) including replication lag.
- ``--json``: Outputs the status in JSON format, ideal for programmatic parsing in monitoring scripts or automation.

3. `ghe-repl-promote` - Promotes a replica instance to become the new primary.

```
ghe-repl-promote [-y --timeout <seconds>]
```

- ``-y``: Assumes "yes" to all prompts, allowing non-interactive promotion. Use with extreme caution in production; ensure you understand the implications.
- ``--timeout <seconds>``: Specifies how long to wait for services to restart during promotion. If services don't become healthy within this timeout, the command might fail.

4. `ghe-repl-stop` - Stops replication between primary and replica.

```
ghe-repl-stop [-y]
```

- ``-y``: Assumes "yes" to all prompts. Stops all replication services. This is typically done before decommissioning a replica or reconfiguring HA.

5. `ghe-backup` - Performs a logical backup of GHES data. (Part of `github/backup-utils`)

```
ghe-backup [-v --skip-git --skip-pages --skip-lfs --config <file>]
```

- `-v`: Verbose output.
- `--skip-git`: Skips backing up Git repository data. Useful if you only need metadata and manage Git data separately (e.g., very large Git data, or specific replication strategies). ****Not recommended for full backups.****
- `--skip-pages`: Skips backing up GitHub Pages data.
- `--skip-lfs`: Skips backing up Git LFS data. If LFS is configured to an external S3 bucket, this is often skipped as S3 provides its own durability.
- `--config <file>`: Specifies an alternative configuration file for `ghe-backup`, overriding the default environment variables or `ghe-backup.config`.

6. `ghe-restore` - Restores a GHES backup to a new instance. (Part of `github/backup-utils`)

```
ghe-restore <backup_directory> [-v --skip-git --skip-pages --skip-lfs --config <file> --force]
```

- `<backup_directory>`: The path to the backup created by `ghe-backup` (e.g., `/mnt/github-backups/github-20230101T000000`).
- `--force`: ****Extremely dangerous.**** Overwrites existing data on the target GHES instance. Only use on a freshly initialized GHES appliance or a throwaway instance.
- Other flags (`-v`, `--skip-git`, etc.) function similarly to `ghe-backup` but apply to the restore operation.

7. `ghe-ssh` - Provides SSH access to the underlying operating system of the GHES appliance.

```
ghe-ssh [-p <port>]
```

- `-p <port>`: Specifies the SSH port. GHES typically uses port 22 for admin SSH.
- This command is critical for advanced diagnostics, inspecting logs, running OS-level commands (e.g., `top`, `df`, `iostat`), and accessing the underlying file system. Be cautious as direct OS modifications are not officially supported and can void warranties or lead to unstable states if not done properly.

8. `gh` CLI (GitHub CLI for Enterprise Admins)

```
gh api -H "Accept: application/vnd.github.v3+json" /enterprises/<enterprise_slug>/settings/members
```

- `gh api`: Accesses GitHub REST API endpoints directly.
- `-H "Accept: application/vnd.github.v3+json"`: Sets the `Accept` header to specify the API version. Essential for ensuring correct response format.
- `/enterprises/<enterprise_slug>/settings/members`: The specific API endpoint to query. This allows programmatic interaction with GHES or GHEC for tasks like listing enterprise members, managing organizations, or checking audit logs.

These commands form the backbone of GHES SRE, enabling setup, monitoring, failover, backup, and advanced diagnostics.

8. Production configuration examples

1. GitHub Enterprise Server (GHES) Management Console Settings (Conceptual)

These are configured via the web-based Management Console (`https://<GHES_HOSTNAME>:8443`).

- **High Availability:**
- **Mode:** `Active/Passive`
- **Primary Hostname:** `github.company.com`
- **Replica Hostname:** `github-replica.company.com` (or IP)
- **Replication Status:** Monitored via dashboard.
- **Authentication:**
- **Method:** `SAML SSO`
- **IdP SSO URL:** `https://idp.company.com/sso`
- **Issuer:** `github.company.com`
- **Certificate:** Upload IdP's public certificate.
- **Signature Method:** `SHA256`
- **Admin Sync:** Enabled, mapping IdP groups (e.g., `GitHub-Admins`) to GHES site administrators.
- **Monitoring:**
- **Prometheus Metrics:** Enabled, with a strong, rotated authentication token.
- **Log Forwarding:** `Syslog-NG` to `syslog.central-logging.company.com:514` (TCP) with `TLS` enabled and client certificate authentication. Forwarding `Authentication`, `Audit`, `Console`, `Hook`, `Nginx`, `Production`, `System` logs.
- **Pre-receive Hooks:**
- **Enabled:** `true`
- **Global Pre-receive Hook:** `Require Signed Commits` (pointing to a script in a designated repository).
- **Global Pre-receive Hook:** `Block PII in Commits` (pointing to a regex-matching script).
- **Git LFS:**
- **External Storage:** `Amazon S3` or `MinIO`
- **Bucket Name:** `github-lfs-data-prod`
- **Region:** `us-east-1`
- **Access Key ID:** `AKIA...` (Use IAM roles if possible)
- **Secret Access Key:** `...` (Use temporary credentials/IAM roles)
- **SMTP:**
- **Server Address:** `smtp.company.com`
- **Port:** `587`
- **TLS:** `true`
- **Authentication:** `Login`
- **Username:** `github@company.com`

- **Password:**`...`

2. `ghe-backup.config` for Automated Backups

```
# /etc/ghe-backup.config (on the dedicated backup host)

# REQUIRED:
# GHE_HOSTS: List of GitHub Enterprise hosts to back up.
# For HA, backup from the active primary hostname.
GHE_HOSTS="github.company.com"

# GHE_DATA_DIR: The local directory where backups will be stored before offloading.
GHE_DATA_DIR="/mnt/github-backups"

# GHE_SSH_USER: The SSH user for connecting to the GHES instance. Typically 'admin'.
GHE_SSH_USER="admin"

# GHE_SSH_PORT: The SSH port for connecting to the GHES instance. Default is 122.
GHE_SSH_PORT=122

# OPTIONAL:
# S3_BUCKET: S3-compatible bucket URL for offloading backups.
# Example: s3://my-github-backup-bucket/prod
S3_BUCKET="s3://github-enterprise-prod-backups/daily"

# S3_ENDPOINT: Custom S3 endpoint if not AWS or a specific region.
# Example: https://s3.us-west-2.amazonaws.com or https://minio.company.com
S3_ENDPOINT="https://s3.us-east-1.amazonaws.com"

# AWS_ACCESS_KEY_ID & AWS_SECRET_ACCESS_KEY: For S3 authentication.
# For production, prefer IAM roles attached to the backup host.
# If using keys, ensure they have minimal necessary permissions (PutObject, GetObject,
DeleteObject, ListBucket).
# AWS_ACCESS_KEY_ID="AKIAxxxxxxxxxxxxxxxx"
# AWS_SECRET_ACCESS_KEY="xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx"

# Retention Policies (for local and S3 backups)
GHE_KEEP_DAILY_BACKUPS=7    # Keep 7 daily backups
GHE_KEEP_WEEKLY_BACKUPS=4   # Keep 4 weekly backups
GHE_KEEP_MONTHLY_BACKUPS=12 # Keep 12 monthly backups

# Compression level for backups (0-9, 0=no compression, 9=max compression)
GHE_COMPRESSION_LEVEL=6

# Bandwidth limit for rsync (KB/s). Adjust based on network capacity.
# GHE_RSYNC_BWLIMIT=50000 # 50 MB/s

# Flags to skip specific data types (use with caution, ensure separate handling if skipped)
```

```
# GHE_SKIP_GIT="true"
# GHE_SKIP_PAGES="true"
# GHE_SKIP_LFS="true" # Often skipped if LFS is on external S3 with its own retention
```

3. GitHub Actions Self-Hosted Runner Configuration (`.runner` file)

This file is generated when registering a runner, but its contents can be inspected or modified for specific needs.

```
# .runner (located in the runner's root directory, typically ~/.runner)

# This file is critical for runner registration and operation.
# It should be protected with appropriate filesystem permissions (e.g., owner-only
read/write).

# Runner ID assigned by GitHub
id: 12345

# Name of the runner
name: "my-ephemeral-runner-001"

# URL of the GitHub server (GHES instance)
serverUrl: "https://github.company.com"

# URL of the GitHub Actions server (often same as serverUrl for GHES)
actionsUrl: "https://github.company.com/_apis/vnext/actions"

# Current runner version
version: "2.308.0"

# Labels assigned to the runner for job targeting
labels:
  - "self-hosted"
  - "linux"
  - "x64"
  - "production"
  - "custom-toolchain-v2"

# Runner operating system
os: "Linux"

# Runner architecture
arch: "x64"

# Path to the runner working directory
workFolder: "_work"

# Whether the runner is configured to be ephemeral (true for auto-scaling scenarios)
ephemeral: true
```

Security Note: The `.credentials` file (containing the runner's PAT for GHES) is a sibling to `.runner` and is highly sensitive. It must be secured with strict filesystem permissions (e.g., `chmod 600 .credentials`). For ephemeral runners, this PAT should be short-lived or tied to an IAM role.

4. Example Pre-receive Hook Script (e.g., `enforce_signed_commits.sh`)

This script resides in a special repository (e.g., `pre-receive-hooks`) configured in the GHES Management

Console.

```
#!/usr/bin/env bash
# enforce_signed_commits.sh
# This pre-receive hook ensures all commits being pushed are cryptographically signed.

# GHES provides these environment variables:
# $GIT_DIR: Path to the Git repository.
# $GL_REPOSITORY: The repository name in the format 'owner/repo-name'.
# $GL_PROTOCOL: The protocol used for the push (ssh, http, https).
# $GL_USER: The username of the pusher.

# Read new commits from standard input (stdin)
while read oldrev newrev refname; do
    # Skip branch deletion
    [ "$newrev" = "0000000000000000000000000000000000000000" ] && continue

    # Get all new commits introduced by this push for the current refname
    # Use --not "$oldrev" to get only new commits relative to the old revision.
    # Use --first-parent to only check the merge history of the pushed branch, not all
    # divergent branches.
    NEW_COMMITS=$(git rev-list --pretty=format:"%H %G? %GS" --first-parent "$oldrev".."$newrev"
    --)

    # Iterate through each new commit to check for signed status
    echo "$NEW_COMMITS" | while read commit_hash signed_status signing_key; do
        if [ -z "$commit_hash" ]; then
            continue # Skip empty lines
        fi

        # Check the signed_status: 'G' means Good signature, 'U' means Unknown (unsigned)
        # The %G? format specifier returns 'G' for good, 'B' for bad, 'U' for unsigned, 'X' for
        # unknown, 'E' for expired
        if [ "$signed_status" != "G" ]; then
            echo
            "-----" >&2
            echo "ERROR: Commit $commit_hash by $GL_USER is not cryptographically signed." >&2
            echo "All commits pushed to this repository MUST be signed using a GPG key." >&2
            echo "Please rebase your branch, sign your commits, and try again." >&2
            echo "Example: git commit -S -m 'Signed commit message'" >&2
            echo
            "-----" >&2
            exit 1 # Reject the push
        fi
    done
done

exit 0 # Allow the push if all commits are signed
```

9. Security considerations & Hardening Best Practices

Operating GitHub in a production enterprise environment requires stringent security measures.

1. Network Isolation and Segmentation:

- **Private Subnets:** Deploy all GHES instances (primary, replica, geo-replica, backup host) and self-hosted runners in private subnets without direct internet access.
- **Security Groups/Network ACLs:** Implement least-privilege network rules.
- **GHES Instances:**
 - Inbound: HTTPS (8443) from Load Balancer, SSH (22, 122) from bastion host/management network, internal replication ports (e.g., 8080-8443, 1194, 22) from other GHES instances.
 - Outbound: To Identity Provider (LDAP/SAML), SMTP server, DNS, S3-compatible storage, external monitoring/logging systems.
- **Self-hosted Runners:**
 - Inbound: None (or specific ports if GHES needs to initiate connection, but typically runners initiate outbound).
 - Outbound: To GHES instance (HTTPS 443), artifact repositories, internal build dependencies, secrets manager. Strictly control egress to prevent data exfiltration.
- **Web Application Firewall (WAF):** Place a WAF (e.g., Cloudflare, AWS WAF, Nginx with ModSecurity) in front of the GHES Load Balancer to protect against common web vulnerabilities (OWASP Top 10) and DDoS attacks.

2. Identity and Access Management (IAM):

- **External Authentication:** Integrate GHES with an enterprise Identity Provider (SAML/LDAP/OIDC) for centralized user management. Disable local authentication where possible.
- **SSO Enforcement:** Enforce SSO for all organizations and repositories.
- **Site Admin Roles:** Grant GHES site administrator privileges sparingly and review regularly. Map to specific IdP groups.
- **Least Privilege:** Configure repository and organization permissions (read, triage, write, maintain, admin) based on the principle of least privilege. Use teams for easier management.
- **Audit Logging:** Enable comprehensive audit logging on GHES and forward to a SIEM. Monitor for suspicious login attempts, permission changes, repository deletions.
- **SSH Key Management:** Regularly review SSH keys configured for users and `ghe-admin`. Enforce strong key policies.

3. Encryption:

- **Data at Rest:** Ensure all block storage volumes (for Git data, databases) are encrypted (e.g., AWS EBS encryption, Azure Disk Encryption). Object storage for LFS and backups should also be encrypted (e.g., S3 SSE-KMS/SSE-S3).

- **Data in Transit:** Enforce TLS 1.2+ for all communications:
 - Between user/LB and GHES.
 - Between GHES instances for replication.
 - Between GHES and external services (IdP, SMTP, monitoring).
 - Between self-hosted runners and GHES.
- **Secrets:** GHES secrets (e.g., PATs for integrations, webhook secrets) are encrypted at rest. For self-hosted runners, integrate with external secrets managers (e.g., AWS Secrets Manager, HashiCorp Vault) to inject ephemeral credentials.

4. Patch Management & Upgrades:

- **Regular Updates:** Keep GHES updated with the latest security patches and versions. GitHub regularly releases security advisories.
- **Staging Environment:** Test GHES upgrades in a non-production environment before applying to production.
- **Automated Patching:** Automate OS-level patching on GHES instances (if using custom images) and self-hosted runners, but carefully manage reboots.

5. Self-Hosted Runner Security:

- **Ephemeral Runners:** Always deploy runners as ephemeral instances. Terminate and replace them after each job to prevent state contamination and reduce the attack surface.
- **Dedicated Environment:** Run runners in a dedicated, isolated environment (e.g., separate Kubernetes cluster, dedicated auto-scaling group) with strict network policies.
- **Minimal Image:** Use a minimal, hardened base image for runners, containing only necessary tools. Regularly scan images for vulnerabilities.
- **Secrets Management:** Never hardcode secrets in runner images or scripts. Use OIDC, IAM roles, or a secrets manager for runtime credential injection.
- **Resource Limits:** Implement resource limits (CPU, memory, disk) to prevent denial-of-service attacks or resource exhaustion.

6. Pre-receive Hooks:

- Implement pre-receive hooks to enforce security policies at the Git level, such as:
- Requiring signed commits (GPG).
- Blocking commits containing sensitive information (e.g., regex for PII, API keys).
- Enforcing branch naming conventions or commit message formats.

7. Regular Audits and Scans:

- **Vulnerability Scanning:** Periodically scan GHES instances and runner images for vulnerabilities.
- **Penetration Testing:** Conduct regular penetration tests of the GHES environment.
- **Configuration Drift:** Monitor GHES configuration for unauthorized changes.

By meticulously applying these hardening best practices, enterprises can significantly reduce the attack surface and enhance the overall security posture of their GitHub environment.

10. Observability & Monitoring considerations

Comprehensive observability is paramount for maintaining a healthy and performant GitHub Enterprise Server instance.

Prometheus Metrics to Watch

GHES exposes a rich set of Prometheus metrics. These should be scraped regularly (e.g., every 15-30 seconds) and visualized in Grafana.

1. System Health (via ``node_exporter`` on GHES instances):

- ``node_cpu_seconds_total``: CPU utilization (idle, user, system). High ``user`` or ``system`` CPU indicates heavy load or kernel issues.
- ``node_memory_MemAvailable_bytes``: Available memory. Low values indicate memory pressure, leading to swapping.
- ``node_filesystem_avail_bytes``: Disk space available on critical partitions (``/``, ``/data``). Low space can halt operations.
- ``node_network_receive_bytes_total``, ``node_network_transmit_bytes_total``: Network I/O. Spikes might indicate heavy Git traffic or network issues.
- ``node_load1``, ``node_load5``, ``node_load15``: System load average. Sustained high loads suggest resource bottlenecks.

2. GHES Application Health (via ``/api/v3/enterprise/metrics``):

- **Replication Status (for HA/DR):**
- ``github_replication_status_health{gh_instance_type="replica"}``: ``1`` for healthy, ``0`` for unhealthy. **Critical alert.**
- ``github_replication_status_lag_seconds{gh_instance_type="replica"}``: Replication lag in seconds. Spikes indicate issues.
- ``github_replication_status_percent_done{gh_instance_type="replica"}``: Percentage of initial sync complete.
- **HTTP Requests:**
- ``github_http_requests_total``: Total HTTP requests to GHES. Track rate and count.
- ``github_http_requests_duration_seconds_bucket``: Latency of HTTP requests. Monitor 90th, 95th, 99th percentiles.
- ``github_http_requests_status_code_total{status_code=~"5..|429"}``: Count of 5xx errors (server-side issues) and 429s (rate limiting). **Critical alert on 5xx increase.**
- **Git Operations:**
- ``github_git_operations_total``: Total Git operations (push, pull, clone).
- ``github_git_operations_duration_seconds_bucket``: Latency of Git operations.

- ``github_git_operations_failed_total``: Count of failed Git operations. ****Alert on increase.****
- ****Job Queues (Sidekiq/Redis):****
- ``github_job_queues_length``: Length of various background job queues. Sustained high values indicate backlog, potential performance degradation for webhooks, notifications, etc. ****Alert if consistently increasing.****
- ``github_job_queues_processed_total``: Number of jobs processed.
- ``github_job_queues_failed_total``: Number of failed jobs.
- ****Database (PostgreSQL):****
- ``github_database_connections_total``: Active database connections.
- ``github_database_query_duration_seconds_bucket``: Database query latency.
- ``github_database_errors_total``: Database errors.
- ****Elasticsearch (Search):****
- ``github_elasticsearch_query_duration_seconds_bucket``: Search query latency.
- ``github_elasticsearch_indexing_latency_seconds_bucket``: Indexing latency.
- ****Git LFS:****
- ``github_lfs_requests_total``: Total LFS requests.
- ``github_lfs_errors_total``: LFS operation errors.

3. Self-Hosted Runner Metrics:

- ``github_actions_runner_total{status="online"}``: Number of online runners. ****Alert if below desired threshold.****
- ``github_actions_runner_busy_total``: Number of busy runners.
- ``github_actions_jobs_queued_total``: Number of pending jobs. High value indicates insufficient runners.
- ``node_cpu_seconds_total``, ``node_memory_MemAvailable_bytes`` (from ``node_exporter`` on runner VMs): Resource utilization of individual runners.

Log Aggregation

GHEES generates various log types that are crucial for diagnostics. All logs should be forwarded to a centralized log aggregation system (e.g., Splunk, ELK Stack, Loki) for indexing, searching, and alerting.

- ****Audit Logs:**** Records administrative actions, security events (logins, permission changes, repository creation/deletion). ****Critical for security monitoring and compliance.****
- ****Authentication Logs:**** Detailed logs related to user authentication via SAML, LDAP, or local accounts.
- ****Production Logs:**** Application-level logs from the GHEES web application. Contains errors, warnings, and general application flow.
- ****Nginx Access/Error Logs:**** Web server access and error logs, useful for identifying client-side issues, broken links, or blocked requests.
- ****Git Access Logs:**** Records all Git operations (clone, fetch, push) including user, repository, and IP.
- ****Hook Logs:**** Detailed logs about webhook deliveries and their status (success/failure, response times).

- ****Console Logs:**** System-level messages from the GHES appliance.
- ****Replication Logs:**** Detailed logs about the `ghe-repl` process.

Alerting Strategy (using Prometheus Alertmanager):

Define clear Service Level Objectives (SLOs) for GHES availability, latency, and error rates. Create alerts that fire when these SLOs are violated or when critical operational thresholds are crossed.

Example Prometheus Alerting Rules (`github\_alerts.yml`):

```
groups:
  - name: github-enterprise-alerts
    rules:
      # --- GHES System Health Alerts ---
      - alert: GHEHighCPULoad
        expr: avg by (instance_ip, gh_instance_type) (node_load5 / count without(cpu)
(node_cpu_seconds_total{mode="idle"})) > 0.8 # 80% CPU utilization over 5 mins
        for: 5m
        labels:
          severity: warning
          tier: critical
        annotations:
          summary: "GitHub Enterprise instance {{ $labels.instance_ip }} ({{
$labels.gh_instance_type }}) CPU load is high"
          description: "The 5-minute load average on GitHub Enterprise instance {{
$labels.instance_ip }} ({{ $labels.gh_instance_type }}) has exceeded 80% of available CPU
cores for more than 5 minutes. This may indicate performance degradation or resource
exhaustion."

      - alert: GHEDiskSpaceLow
        expr: (node_filesystem_avail_bytes{mountpoint="/data",fstype!="rootfs"} /
node_filesystem_size_bytes{mountpoint="/data",fstype!="rootfs"}) * 100 < 10
        for: 10m
        labels:
          severity: critical
          tier: critical
        annotations:
          summary: "GitHub Enterprise instance {{ $labels.instance_ip }} ({{
$labels.gh_instance_type }}) disk space is low"
          description: "The '/data' partition on GitHub Enterprise instance {{
$labels.instance_ip }} ({{ $labels.gh_instance_type }}) has less than 10% disk space
remaining. Operations may fail or be degraded soon."

      # --- GHES Replication Alerts (for HA/DR) ---
      - alert: GHEReplicationUnhealthy
        expr: github_replication_status_health{gh_instance_type="replica"} == 0
        for: 2m
        labels:
          severity: critical
```

```
    tier: critical
  annotations:
    summary: "GitHub Enterprise HA/DR Replica is unhealthy"
    description: "The GitHub Enterprise replica instance ({{ $labels.instance_ip }}) is reporting an unhealthy replication status. Failover may not be possible."

- alert: GHEReplicationLagging
  expr: github_replication_status_lag_seconds{gh_instance_type="replica"} > 600 # 10 minutes lag
  for: 5m
  labels:
    severity: warning
    tier: critical
  annotations:
    summary: "GitHub Enterprise HA/DR Replica is lagging"
    description: "The GitHub Enterprise replica instance ({{ $labels.instance_ip }}) has a replication lag of more than 10 minutes. This could impact RPO during failover."

# --- GHES Application Error Alerts ---
- alert: GHEHighHTTPErrorRate
  expr: sum(rate(github_http_requests_status_code_total{status_code=~"5.."}[5m])) by (gh_instance_type) > 5 # More than 5 5xx errors per second
  for: 1m
  labels:
    severity: critical
    tier: critical
  annotations:
    summary: "GitHub Enterprise {{ $labels.gh_instance_type }} experiencing high HTTP 5xx error rate"
    description: "The GitHub Enterprise {{ $labels.gh_instance_type }} instance is serving more than 5 HTTP 5xx errors per second, indicating a severe application issue."

- alert: GHEHighAPIRateLimitErrors
  expr: sum(rate(github_http_requests_status_code_total{status_code="429"}[5m])) by (gh_instance_type) > 10 # More than 10 429 errors per second
  for: 5m
  labels:
    severity: warning
    tier: important
  annotations:
    summary: "GitHub Enterprise {{ $labels.gh_instance_type }} experiencing high API rate limit errors"
    description: "The GitHub Enterprise {{ $labels.gh_instance_type }} instance is responding with an increasing number of 429 (Too Many Requests) errors, indicating integrations might be hitting API rate limits."

# --- GHES Job Queue Alerts ---
- alert: GHEJobQueueBacklog
```

```

    expr: sum(github_job_queues_length) by (gh_instance_type) > 1000 # Total queue length
exceeds 1000 jobs
    for: 10m
    labels:
      severity: critical
      tier: critical
    annotations:
      summary: "GitHub Enterprise {{ $labels.gh_instance_type }} job queues are backing
up"
      description: "The total length of background job queues on GitHub Enterprise {{
$labels.gh_instance_type }} has exceeded 1000 jobs for 10 minutes, indicating a potential
bottleneck or worker issue. Webhook delivery, notifications, etc., may be delayed."

# --- GitHub Actions Self-Hosted Runner Alerts ---
- alert: GHESelfHostedRunnersOffline
    expr: sum(github_actions_runner_total{status="online"}) by (gh_instance_type) < 1 #
No online runners
    for: 5m
    labels:
      severity: critical
      tier: important
    annotations:
      summary: "All GitHub Actions self-hosted runners for {{ $labels.gh_instance_type }}
are offline"
      description: "There are no online GitHub Actions self-hosted runners registered to
GitHub Enterprise {{ $labels.gh_instance_type }}. CI/CD jobs will not execute."

- alert: GHESelfHostedRunnersBelowThreshold
    expr: sum(github_actions_runner_total{status="online"}) by (gh_instance_type) < 5 #
Example: fewer than 5 runners
    for: 10m
    labels:
      severity: warning
      tier: important
    annotations:
      summary: "GitHub Actions self-hosted runners count below threshold for {{
$labels.gh_instance_type }}"
      description: "The number of online GitHub Actions self-hosted runners for GitHub
Enterprise {{ $labels.instance }} is below the configured threshold (5). CI/CD job throughput
may be impacted."

- alert: GHEActionsJobsQueuedHigh
    expr: sum(github_actions_jobs_queued_total) by (gh_instance_type) > 50
    for: 5m
    labels:
      severity: warning
      tier: important
    annotations:

```

```
summary: "High number of queued GitHub Actions jobs for {{ $labels.gh_instance_type
}}"
description: "There are more than 50 jobs queued for GitHub Actions on GitHub
Enterprise {{ $labels.gh_instance_type }}. This suggests insufficient runner capacity or slow
job execution."
```

11. Common troubleshooting scenarios with RCA (Root Cause Analysis) steps

Scenario 1: GHES UI Unresponsive / High Latency

Symptoms: Users report slow UI, timeouts, Git pushes/pulls failing or extremely slow. ``ghe-repl-status`` might show issues.

Troubleshooting Steps:

1. Check System Resources (``ghe-ssh``):

- ``top`` / ``htop``: Look for high CPU utilization (which processes?), high memory usage (swapping?), ``kswapd`` activity.
- ``df -h``: Check disk space, especially ``/data``.
- ``iostat -xz 5``: Check disk I/O (await, %util, r/s, w/s). High ``await`` or ``%util`` on ``/data`` indicates storage bottleneck.
- ``netstat -s``: Look for network errors, dropped packets.

2. Check GHES Internal Services:

- ``ghe-status``: Overview of all GHES services. Look for services in ``down`` or ``unhealthy`` state.
- ``ghe-service-status <service_name>``: Detailed status for specific services (e.g., ``github-unicorn``, ``github-app-sidekiq``, ``github-git-http``, ``github-postgresql``).
- ``tail -f /var/log/github/github.log`` / ``grep ERROR /var/log/github/*``: Look for recent errors in application logs.
- ``ghe-support-bundle``: Generate a support bundle for deep analysis (contains logs, configs, diagnostics).

3. Network Check:

- ``ping``, ``traceroute`` from GHES to Load Balancer, DNS, IdP, S3.
- Check Load Balancer health checks: Is it marking the instance as unhealthy? Why?

4. Database Health:

- ``ghe-dbconsole``: Access PostgreSQL console. Check for long-running queries, locks. ``SELECT pid, age(query_start, clock_timestamp()), username, query FROM pg_stat_activity WHERE state != 'idle';``
- Check ``github_database_connections_total`` and ``github_database_query_duration_seconds`` in Prometheus.

Common RCA & Solutions:

- **High CPU/Memory:**
- **RCA:** Misconfigured background jobs, a "noisy neighbor" process, runaway web requests (e.g., large-scale API scraping without proper pagination), or insufficient instance sizing.
- **Solution:** Increase instance size, optimize problematic queries/integrations, identify and throttle abusive API clients, restart problematic services (`ghe-service restart <service>`).
- **Disk I/O Bottleneck:**
- **RCA:** Under-provisioned block storage (IOPS/throughput limits), heavy Git LFS usage without external S3, or intensive indexing operations.
- **Solution:** Upgrade disk type (e.g., GP2 to GP3 on AWS, increase provisioned IOPS), ensure Git LFS is correctly offloaded to S3, schedule large indexing during off-peak hours.
- **Database Issues:**
- **RCA:** Long-running queries blocking others, connection pool exhaustion, inefficient schema.
- **Solution:** Identify and kill rogue queries (`pg_cancel_backend`), optimize application code, tune PostgreSQL parameters via GHES Management Console (e.g., `shared_buffers`, `work_mem`).

Scenario 2: GHES Replication Lag

Symptoms: `ghe-repl-status` shows increasing lag for Git, MySQL, or Elasticsearch. Failover would result in data loss.

Troubleshooting Steps:

1. Check Network Connectivity (`ghe-ssh` on both primary and replica):
 - `ping <primary_ip>` / `ping <replica_ip>`: Check basic reachability.
 - `mtr <primary_ip>` / `mtr <replica_ip>`: Check latency and packet loss between instances.
 - Verify Security Group/Network ACL rules allow ports 8080-8443, 1194, 22 between primary and replica.
2. Check Resource Utilization on Both Instances: Follow steps from Scenario 1. If primary is overloaded, it might struggle to send replication data. If replica is overloaded, it might struggle to apply it.
3. Check Replication Processes:
 - `ps aux | grep ghe-repl`: Ensure replication processes are running on both instances.
 - Check `/var/log/github/resque/replication.log` for errors.
4. Storage Performance:
 - `iostat` on both primary and replica. High write latency on the primary or high read/write latency on the replica can cause lag.
 - Ensure storage types (IOPS/throughput) are identical and sufficient for both primary and replica.

Common RCA & Solutions:

- **Network Latency/Packet Loss:**
- **RCA:** Underlying cloud network issues, misconfigured routing, insufficient bandwidth.
- **Solution:** Contact cloud provider, optimize network paths (e.g., dedicated interconnects), ensure instances are in close proximity (same AZ).

- **Resource Bottleneck (Primary or Replica):**
- **RCA:** Primary is too busy to push changes, or replica is too slow to apply them (CPU, memory, disk I/O).
- **Solution:** Scale up instances, identify and optimize heavy workloads, tune storage.
- **Large Data Volume:**
- **RCA:** Extremely large pushes to Git or database changes cause a temporary spike in data to replicate.
- **Solution:** Monitor and anticipate. Consider fine-tuning `ghe-repl` parameters (advanced).

Scenario 3: Self-Hosted GitHub Actions Runner Offline/Stuck

Symptoms: Runners disappear from GHES UI, jobs stay queued, or jobs get stuck indefinitely.

Troubleshooting Steps:

1. Check Runner Host Status:

- SSH to the runner VM/node.
- `systemctl status actions.runner.<org>.<runner\_name>.service` (or equivalent for your init system): Check if the runner agent service is running.
- `top`, `df -h`: Check resource utilization on the runner host.
- `netstat -tulnp | grep <runner\_pid>`: Ensure the runner process has outbound connectivity.
- `tail -f /home/actions-runner/\_log/\*`: Check runner agent logs for errors connecting to GHES, or during job execution.

2. Network Connectivity to GHES:

- `curl -vvv https://github.company.com`: Test connectivity from the runner to GHES. Check for SSL/TLS issues, firewalls.

3. GHES Status:

- Is GHES itself healthy? (Refer to Scenario 1). If GHES is down, runners can't connect.
- Check GHES `Hook Logs` or `Production Logs` for errors related to runner registration/communication.

4. Auto-scaling Group/Kubernetes Health:

- Check logs/events of your auto-scaling controller (e.g., AWS ASG, K8s Cluster Autoscaler) for errors provisioning new instances/pods.
- Are there enough resources in the cluster/account to provision new runners?
- Are max limits hit?

Common RCA & Solutions:

- **Runner Process Crash/Resource Exhaustion:**
- **RCA:** OOM kill, unhandled exception in runner agent, job consumed all resources.
- **Solution:** Ensure `systemd` (or equivalent) is configured to restart the runner service. Implement robust resource limits for jobs (e.g., cgroups, K8s limits). Debug job that caused the crash.
- **Network Issues:**

- **RCA:** Firewall blocking outbound connection to GHES, DNS resolution failure on runner host, proxy issues.
- **Solution:** Verify network security groups/ACLs, check `/etc/resolv.conf`, configure proxy settings for the runner.
- **Stale Runner Token/Configuration:**
- **RCA:** Runner's authentication token expired or was revoked. Configuration drift.
- **Solution:** Re-register the runner, ensure automated provisioning re-generates fresh tokens. For ephemeral runners, this is less of an issue as they are short-lived.
- **GHES Unavailability:**
- **RCA:** GHES itself is down or performing poorly.
- **Solution:** Address GHES issues first (Scenario 1).

12. Common mistakes and how to avoid them in production

1. Under-provisioning GHES Resources:

- **Mistake:** Deploying GHES with insufficient CPU, RAM, or I/O for current and anticipated load. Leads to constant high CPU, memory swapping, disk I/O bottlenecks, and slow performance.
- **Avoid:**
- **Baseline & Plan:** Understand your organization's user count, number of repositories, Git LFS usage, and CI/CD activity. Consult GitHub's sizing guidelines.
- **Monitor Aggressively:** Use Prometheus/Grafana to track `node_cpu`, `node_memory`, `node_disk_io`, `github_http_requests_total`, `github_git_operations_total` from day one. Scale up proactively.
- **Performance Testing:** Conduct load tests simulating peak usage before going live or after significant growth.

2. Ignoring Replication Health and Testing Failovers:

- **Mistake:** Setting up HA/DR replication and assuming it always works. Neglecting to monitor replication lag or never performing a full failover drill. In a real incident, failover fails or results in data loss.
- **Avoid:**
- **Monitor `ghe-repl-status`:** Implement critical alerts for `github_replication_status_health` (unhealthy) and `github_replication_status_lag_seconds` (lagging).
- **Regular Failover Drills:** Schedule and execute full primary-to-replica failover tests (and replica-to-primary failback) at least quarterly. Document the process in a runbook.
- **DR Drills:** Annually, perform a full geo-replica DR test to ensure RTO/RPO objectives are met.

3. Poorly Managed Self-Hosted Runners:

- **Mistake:** Manually maintaining runners, not using auto-scaling, allowing stale runners to persist, or exposing runners with excessive network permissions. Leads to job backlogs, security vulnerabilities, and resource waste.

- **Avoid:**
- **Automate Everything:** Use auto-scaling groups (VMs) or Kubernetes (pods) for ephemeral runners. Deploy runners as code (Terraform, Ansible).
- **Ephemeral Design:** Ensure runners are always provisioned fresh for each job and terminated afterwards.
- **Least Privilege Networking:** Implement strict network policies (Security Groups, Network Policies) for runners.
- **Centralized Secrets:** Integrate with a secrets manager for runtime credential injection, avoiding hardcoded secrets.

4. Inadequate Backup Strategy or Untested Restores:

- **Mistake:** Relying solely on HA for data protection (HA is for availability, not data corruption/deletion), or having backups but never testing the restore process.
- **Avoid:**
- **Implement `ghe-backup`:** Set up regular, automated logical backups to durable object storage (S3-compatible).
- **Test Restores:** Periodically (e.g., semi-annually) perform a full `ghe-restore` to a throwaway GHES instance. This validates the backup integrity and refines your recovery runbook.
- **Geo-Redundant Backups:** Store backups in multiple regions or different object storage accounts.

5. Ignoring API Rate Limits in Integrations:

- **Mistake:** Third-party tools or custom scripts aggressively polling GitHub APIs without respecting rate limits. This leads to 429 errors, service degradation, and potential blacklisting.
- **Avoid:**
- **Educate Developers:** Ensure all developers and integration owners are aware of GitHub's API rate limits (both GHES and GitHub.com).
- **Implement Best Practices:** Use webhooks instead of polling where possible. Implement exponential backoff and jitter for API retries. Use GitHub Apps for fine-grained permissions and higher rate limits where appropriate.
- **Monitor 429s:** Alert on increases in `github\_http\_requests\_status\_code\_total{status\_code="429"}` to proactively identify problematic integrations.

6. Lack of Comprehensive Logging and Centralized Monitoring:

- **Mistake:** Relying only on the GHES UI for status or not aggregating logs externally. Makes root cause analysis difficult and reactive.
- **Avoid:**
- **Centralize Logs:** Forward all GHES logs (`syslog-ng`) to a centralized log aggregation system.
- **Prometheus/Grafana:** Configure and maintain a dedicated monitoring stack for GHES.
- **Proactive Alerts:** Set up alerts for critical metrics and log patterns (e.g., 5xx errors, replication lag, disk space, runner offline).

13. Enterprise-level recommendations (performance tuning, caching, connection pooling)

For an enterprise-grade GitHub Enterprise Server deployment, fine-tuning and operational excellence go beyond basic setup.

1. Automated GHES Upgrades and Patching:

- ****Recommendation:**** Fully automate the GHES upgrade process using infrastructure-as-code (Terraform, Ansible) and scripting. This reduces human error, ensures consistency, and allows for rapid application of security patches.
- ****Implementation:**** Maintain a staging environment that mirrors production. Use ``ghe-upgrade`` command, triggered by an automation pipeline. Leverage pre- and post-upgrade hooks for custom checks and validations.

2. Dedicated GHES Monitoring Stack:

- ****Recommendation:**** Do not co-locate GHES monitoring components (Prometheus, Grafana, Alertmanager) on the GHES instances themselves. Use a separate, highly available monitoring stack.
- ****Implementation:**** Deploy Prometheus, Grafana, and Alertmanager in a dedicated Kubernetes cluster or a separate set of VMs. This ensures monitoring remains available even if GHES experiences a critical outage, and prevents monitoring from consuming GHES resources.

3. Centralized Secrets Management for Runners:

- ****Recommendation:**** Avoid storing static credentials or tokens directly on self-hosted runners or in their configuration files.
- ****Implementation:**** Integrate runners with a secrets management solution like HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, or GCP Secret Manager. Use short-lived, dynamically generated credentials (e.g., IAM roles for EC2 instances, Kubernetes Service Account tokens with OIDC for pods) to fetch secrets at job runtime.

4. GHES Management Console APIs for Automation:

- ****Recommendation:**** Leverage the GHES Management Console API for programmatic configuration and automation tasks.
- ****Implementation:**** Automate tasks like adding SSH keys, configuring log forwarding, managing SSL certificates, or setting up external authentication via scripts interacting with the Management Console API, rather than manual UI clicks. This is especially useful for DR recovery scenarios.

5. Comprehensive Runbook for GHES Incidents:

- ****Recommendation:**** Develop and regularly update detailed, actionable runbooks for common GHES incident scenarios (e.g., primary instance failure, replication lag, storage exhaustion, UI unresponsiveness).
- ****Implementation:**** Runbooks should include: Symptoms, Alerting details, Troubleshooting steps (with commands), RCA steps, Resolution steps, Escalation paths, Communication templates. Store runbooks in a Git repository (outside GHES, e.g., in a separate GitLab instance or Confluence) for accessibility during GHES outages.

6. Performance Tuning for Large Repositories and Git LFS:

- **Recommendation:** Optimize Git operations for very large repositories or heavy Git LFS usage.
- **Implementation:**
- **Git LFS Caching Proxies:** For distributed developer teams, consider deploying Git LFS caching proxies closer to developers to reduce latency and bandwidth usage to the central S3 bucket.
- **Partial Clone/Shallow Clone:** Educate developers on using `git clone --filter=blob:none` (partial clone) or git clone --depth 1` (shallow clone) for faster initial clones of large repositories, especially in CI/CD.`
- **Git Config Optimization:** Tune client-side Git configuration (e.g., `git config --global core.compression 0` , git config --global pack.threads 1` ) for performance on build agents.`
- **Repository Pruning/Archiving:** Regularly review and archive inactive or excessively large repositories to reduce GHES load.

7. Database Performance Tuning (via Management Console):

- **Recommendation:** Adjust PostgreSQL parameters based on GHES workload and instance size.
- **Implementation:** Access the GHES Management Console and navigate to the "Performance" or "Database" settings. Consider adjusting:
 - `shared_buffers` : Controls memory allocated for database caches.`
 - `work_mem` : Memory for complex query operations.`
 - `effective_cache_size` : Estimation of available cache memory.`
- Consult GHES documentation and performance metrics (`github_database_query_duration_seconds`) to guide tuning. Be cautious, as incorrect settings can degrade performance.`

8. Connection Pooling (for external applications):

- **Recommendation:** For applications frequently interacting with GitHub APIs, implement robust connection pooling.
- **Implementation:** Use client libraries that support HTTP connection pooling. Configure appropriate pool sizes and timeouts to avoid exhausting GHES resources or hitting OS file descriptor limits. This applies to both custom applications and third-party integrations.

14. Advanced concepts relating to this part

1. GHES Cluster Mode (Experimental/Limited Availability):

- While HA (active/passive) is the standard for GHES, GitHub has explored and offers a "cluster" mode for certain configurations, primarily designed for extreme scale and fault tolerance by distributing services across multiple nodes. This is significantly more complex to deploy and manage than HA.
- **Concept:** Instead of a single primary and one or more replicas, services like Git, MySQL, and Elasticsearch run on multiple independent nodes, providing active-active capabilities and higher resilience within a single region.
- **SRE Impact:** Requires advanced Kubernetes or distributed systems knowledge, complex network configuration, and specialized monitoring for individual service components across the cluster. Not typically the first choice for most enterprises unless very specific scale requirements dictate it.

2. Custom Pre-receive Hooks for Complex Policy Enforcement:

- **Concept:** Go beyond simple script-based checks. Integrate pre-receive hooks with external systems for more dynamic and complex policy enforcement.
- **Implementation:**
 - **API Calls:** A pre-receive hook can make an API call to an internal policy engine (e.g., OPA Gatekeeper, custom microservice) that evaluates commit content, author attributes, or branch protection rules dynamically.
 - **Vulnerability Scanning Integration:** Trigger a lightweight static analysis scan on proposed changes and block the push if critical vulnerabilities are detected (though this can add latency).
 - **Secret Scanning:** Integrate with enterprise secret scanning tools to catch hardcoded secrets before they enter the repository.
- **Challenges:** Performance (hooks add latency), complexity of external system integration, maintaining the policy engine.

3. Hybrid GitHub Deployments (GHES + GitHub.com Enterprise Cloud):

- **Concept:** Some organizations use GHES for highly sensitive, compliance-driven codebases and GitHub.com Enterprise Cloud for less sensitive projects, open-source contributions, or specific teams.
- **SRE Impact:** Managing two distinct GitHub environments. Requires consistent tooling (e.g., `gh` CLI), synchronized identity management (if possible), and potentially a strategy for code migration or cross-platform collaboration. Monitoring and security strategies must adapt to both managed and self-managed environments.

4. Optimizing Git Protocol Performance (Smart HTTP, Partial Clone, Shallow Clone):

- **Concept:** Understanding and leveraging advanced Git features to reduce network traffic and improve client-side performance, especially for large repositories.
- **Smart HTTP:** Modern Git uses HTTP(S) as the primary transport, which is firewall-friendly and performant. GHES heavily relies on this.
- **Partial Clone** (`--filter=blob:none`, `--filter=tree:0`): Clients fetch only the objects necessary for the requested history or branch, reducing initial clone size significantly. GHES supports this.
- **Shallow Clone** (`--depth N`): Limits the history depth, useful for CI/CD builds that only need recent commits.
- **SRE Role:** Ensure GHES is configured to support these features effectively, educate developers and CI/CD engineers, and monitor Git traffic patterns.

5. GitHub Apps for Fine-grained Access and Automation:

- **Concept:** GitHub Apps are the recommended way to build integrations with GitHub. They offer granular permissions, don't act on behalf of a user, and have their own higher rate limits.
- **SRE Impact:** For enterprise integrations (e.g., CI/CD systems, internal bots, security scanners), use GitHub Apps instead of Personal Access Tokens (PATs). This enhances security (least privilege) and improves reliability (higher rate limits, dedicated identity). Managing app registrations and installations becomes an SRE/Ops task.

6. Containerized Self-Hosted Runners with Kubernetes/Nomad:

- **Concept:** Deploying self-hosted runners as ephemeral containers orchestrated by Kubernetes or HashiCorp Nomad, providing superior isolation, scalability, and resource management compared to VM-based runners.
- **Implementation:** Use a `Deployment` or `StatefulSet` for the runner agent, with `Horizontal Pod Autoscaler` (HPA) to scale based on job queue length metrics. Implement `Network Policies` for strict ingress/egress. Leverage `Persistent Volume Claims` for caching dependencies or toolchains.
- **Benefits:** Faster startup times, better resource utilization, declarative infrastructure, easier secret injection (via K8s secrets/Vault integration).

These advanced concepts are crucial for experts to push the boundaries of GitHub's operational efficiency, security, and scalability in complex enterprise environments.

15. Integration with other DevOps tools

GitHub acts as the central hub for code and collaboration, making its integration with other DevOps tools fundamental.

1. CI/CD Systems (Jenkins, GitLab CI, CircleCI, Azure DevOps Pipelines):

- **Triggering Builds:**
- **Webhooks:** GitHub sends webhooks (`push`, `pull_request`, `merge`) to the CI/CD system's endpoint, triggering pipelines.
- **Polling:** Less recommended, but some older systems might poll GitHub for new commits.
- **Status Updates:** CI/CD systems update GitHub commit statuses or pull request checks, providing immediate feedback on build success/failure.
- **Artifact Storage:** CI/CD pipelines often produce artifacts that are stored in artifact repositories (e.g., Nexus, Artifactory) or directly attached to GitHub Releases.
- **Code Coverage/Quality:** Tools like SonarQube integrate by commenting on pull requests with analysis results, leveraging GitHub's PR API.
- **Authentication:** CI/CD systems authenticate to GitHub via Personal Access Tokens (PATs) or GitHub Apps for granular permissions.

2. Terraform (Infrastructure as Code):

- **github Provider:** Terraform's `github` provider allows declarative management of GitHub resources.
- **Use Cases:**
- **Repository Management:** Creating, updating, and deleting repositories.
- **Team Management:** Creating teams, adding/removing members, assigning repository permissions.
- **Branch Protection Rules:** Enforcing policies like requiring signed commits, mandatory reviews, status checks.
- **Webhooks:** Configuring webhooks for CI/CD triggers or other integrations.

- **Organization/Enterprise Settings:** Managing various organization-level settings (though less comprehensive than the GHES Management Console API).
- **Benefits:** Version control for GitHub configuration, auditability, consistency across organizations/repositories.

3. Kubernetes (Container Orchestration):

- **Self-hosted Runners:** Deploying GitHub Actions self-hosted runners as ephemeral pods within a Kubernetes cluster.
- **Deployment:** `Deployment` or `StatefulSet` for the runner agent.
- **Scaling:** `Horizontal Pod Autoscaler` (HPA) to scale runners based on CPU/memory utilization or custom metrics (e.g., `github\_actions\_jobs\_queued\_total`).
- **Resource Management:** Kubernetes resource requests and limits for pods.
- **Network Policies:** Strict network segmentation for runner pods.
- **GitOps:** GitHub repositories serve as the single source of truth for Kubernetes manifest files, which are then applied to clusters by GitOps operators (e.g., Argo CD, Flux CD). A Git push to a specific branch triggers a deployment.
- **Secrets Management:** Kubernetes `Secrets` or integration with external secrets managers (e.g., HashiCorp Vault K8s integration) to provide runner pods with necessary credentials.

4. Ansible (Configuration Management/Orchestration):

- **GHES Configuration:** Automating initial setup and ongoing configuration of GHES instances (e.g., applying security patches, configuring log forwarding, managing SSH keys).
- **Self-hosted Runner Deployment:** Automating the provisioning, installation, and registration of self-hosted runners on VMs or bare-metal servers.
- **Repository Synchronization:** Using Ansible to ensure consistent configurations (e.g., `.gitignore`, `CODEOWNERS`) across multiple repositories.
- **Pre-receive Hook Deployment:** Automating the deployment and updates of pre-receive hook scripts to the GHES designated repository.

5. Monitoring & Logging (Prometheus, Grafana, ELK/Splunk):

- **Metrics:** Prometheus scrapes metrics from GHES (and `node\_exporter` on runners) for real-time performance and health monitoring.
- **Dashboards:** Grafana visualizes these metrics, providing operational dashboards.
- **Alerting:** Alertmanager (integrated with Prometheus) sends notifications based on predefined rules.
- **Logs:** GHES logs are forwarded via `syslog-ng` to centralized logging platforms (Elasticsearch, Logstash, Kibana - ELK; or Splunk, Loki) for deeper diagnostics, auditing, and security analysis.

These integrations are crucial for building a cohesive and automated DevOps toolchain around GitHub, enhancing productivity, security, and operational efficiency.

16. Comparison tables with competing tools

This comparison focuses on the self-managed enterprise versions of code collaboration platforms, specifically from an SRE/operations perspective, as these are the ones where direct infrastructure management and troubleshooting are required.

Feature / Metric (Enterprise Edition)	GitHub Enterprise Server (GHES) Azure DevOps Server	GitLab Self-Managed
Core Focus	Git hosting, collaboration, GitHub Actions CI/CD (Git, CI/CD, Registry, Security, Planning)	End-to-end DevOps platform (Git, CI/CD, Registry, Security, Planning), Boards (Work Item Tracking), Test Plans
HA/DR Capabilities	Active/Passive HA (Primary/Replica). Geo-replication (Async DR). `ghe-backup`/`ghe-restore`. Reference Architectures for HA. Geo-replication (Active/Active or Async DR). SQL AlwaysOn for DB. IIS ARR for Web/App tiers. TFS Backup/Restore.	
Monitoring Hooks	Built-in Prometheus exporter for application metrics. `node_exporter` for OS. Syslog-NG. Built-in Prometheus exporter. Extensive `node_exporter` integration. Syslog. Extensive Azure Monitor/Application Insights integration. Windows Event Logs.	
Manageability	Dedicated Management Console (web UI), `ghe-` CLI tools, APIs. Single appliance model. Helm Charts for Kubernetes. Omnibus package for VMs. Extensive APIs. Windows Server/IIS management. SQL Server Management Studio. TFS Admin Console.	
Scalability	Scales vertically by upgrading instance size. Horizontal scaling for Git LFS and Actions runners. Highly scalable with Kubernetes deployments. Can scale individual services. Scales horizontally with multiple application tiers and SQL clusters.	
CI/CD Solution	GitHub Actions (can use self-hosted runners). Azure Pipelines (can use self-hosted agents).	GitLab CI/CD (built-in, can use self-hosted runners).
Cost Model (Enterprise)	Per-developer licensing. Instance/storage costs are cloud provider specific. Per-developer licensing. Instance/storage costs are cloud provider specific. Per-developer CALs (Client Access Licenses). Instance/storage costs.	
Ease of Deployment	Relatively straightforward appliance deployment (AMI/VHD). HA setup is guided. Omnibus is easy for single node. Kubernetes deployment is complex but robust. Requires Windows Server, SQL Server, IIS expertise for initial setup.	
Container Registry	Packages (separate service from GHES). Azure Artifacts (separate service from ADS).	Built-in Container Registry.
Secret Management Integration	Integrates with external secrets managers for runners. GitHub Secrets. Integrates with external secrets managers (Vault). GitLab Secrets. Integrates with Azure Key Vault. Azure DevOps Variable Groups.	
Git LFS	Integrated. Can offload to S3-compatible storage. Integrated.	Integrated. Can offload to S3-compatible storage.

| Latency Impact (SRE) | HA/DR replication lag is key metric. API latency. | Database/Storage latency critical. CI/CD job execution time. | Database latency. Work item tracking responsiveness.

| Pros | Industry standard for Git. Simple HA setup. Strong ecosystem. | Full DevOps platform. Highly configurable. Excellent K8s integration. | Mature ALM features. Deep integration with Microsoft ecosystem.

| Cons | CI/CD is separate (Actions). Less granular control over internal components. | Can be resource-intensive. Complexity of managing many services. | Windows-centric. Less open-source community support. Legacy feel for some. |

| Typical Use Cases | Code-centric organizations prioritizing Git, PRs, and GitHub Actions. Compliance-heavy. | Organizations seeking an integrated, single-vendor DevOps platform. | Enterprises deeply invested in Microsoft technologies and ALM processes. |

Key SRE Considerations from the Comparison:

- **Operational Overhead:** GHES offers a relatively encapsulated appliance model, simplifying some aspects but potentially limiting fine-grained control. GitLab, especially on Kubernetes, provides immense flexibility but higher operational complexity. Azure DevOps Server ties into Windows Server and SQL Server expertise.
- **Monitoring Depth:** All provide good monitoring, but the *way* you access and interpret it differs. GHES and GitLab lean heavily into Prometheus. Azure DevOps leverages Microsoft's monitoring stack.
- **Failover/Recovery:** All provide HA/DR, but their implementation details (block-level vs. application-level replication, manual vs. automated failover) impact RTO/RPO and the SRE runbook.
- **Scaling:** GHES primarily scales its core components vertically, with horizontal scaling for peripheral services. GitLab and Azure DevOps Server offer more horizontal scaling for their various services.

The choice often comes down to an organization's existing technology stack, operational expertise, feature requirements, and strategic vision for its DevOps toolchain. For SREs, understanding the architectural implications of each choice is paramount.

17. A visual cheat sheet in text/table form

GitHub Enterprise Server (GHES) SRE Cheat Sheet

Category	Item	Description	CLI Command /
Metric / Log		Action / RCA Tip	
System Status	GHES Overall Status	Health of all core services.	
`ghe-status`		First check for any 'down' services.	

	CPU Load	Average CPU utilization.		`node_load1`, `node_load5`, `node_cpu_seconds_total`		High load -> check `top`, `ghe-service-status`.	
	Disk Space	Available space on `/data` partition.		`node_filesystem_avail_bytes{mountpoint="/data"}`		Low space -> `df -h`, identify large files.	
	Memory Usage	Free memory, swapping activity.		`node_memory_MemAvailable_bytes`		Low memory -> `top`, `dmesg` (OOM kills).	
	Replication	HA Replication Status		Health & lag of primary-replica sync.		`ghe-repl-status -v`, `github_replication_status_health`	
						Lag -> check network, disk I/O on both instances.	
	Promote Replica	Initiate failover to replica.		`ghe-repl-promote -y`		Use during primary failure. Test regularly.	
	Backups	Backup Execution		Status and completion of `ghe-backup`.		`ghe-backup -v`, Check S3 bucket for files.	
						Verify cron job, S3 access. Test restore.	
	Restore GHES	Restore from a logical backup.		`ghe-restore <backup_dir> --force`		DANGER! Use on new instance only.	
	Logs	Application Logs		GHES web app errors, warnings.		`/var/log/github/github.log`, `production.log`	
						`grep ERROR`, `tail -f` for real-time issues.	
	Audit Logs	Security-relevant events (logins, permissions).		`/var/log/github/audit.log`		Critical for security and compliance. Forward to SIEM.	
	Replication Logs	Details of `ghe-repl` process.		`/var/log/github/resque/replication.log`		Debugging replication issues.	
	Git Operations	Git Operation Failures		Errors during push/pull/clone.		`github_git_operations_failed_total`	
						Check permissions, storage, network.	
	Git LFS Errors	Issues with Large File Storage.		`github_lfs_errors_total`		Check S3 connectivity/permissions.	
	Web/API	HTTP 5xx Errors		Server-side errors for web UI/API.		`github_http_requests_status_code_total{status_code=~"5.."}`	
						Indicates core application issues.	
	API Rate Limit Errors	`429` (Too Many Requests) responses.		`github_http_requests_status_code_total{status_code="429"}`		Identify and optimize misbehaving integrations.	
	Job Queues	Background Job Queue Length		Backlog of tasks (webhooks, notifications).		`github_job_queues_length`	
						Sustained high -> worker issues, resource starvation.	
	Runners	Self-Hosted Runners Online		Count of available GitHub Actions runners.		`github_actions_runner_total{status="online"}`	
						Alert if below threshold. Check runner host health.	
	Queued Jobs	Number of jobs waiting for runners.		`github_actions_jobs_queued_total`		High queue -> scale runners, optimize jobs.	
	Admin Access	SSH to GHES		Access underlying OS for diagnostics.			

`ghe-ssh`		Advanced troubleshooting. Use with caution.	
	DB Console	Access PostgreSQL for direct queries.	
`ghe-dbconsole`		Identify long-running queries, locks.	

18. A comprehensive final learning summary

Congratulations on reaching the final part of this GitHub study guide, focusing on the critical aspects of Production SRE, Diagnostics, Troubleshooting, and Incident Response for GitHub. The journey from basic usage to becoming an operational expert requires a shift in mindset: from simply *using* GitHub to *ensuring* its unwavering reliability and performance as a foundational platform.

Here's a recap of the key takeaways for your path to becoming an industry expert:

1. **GitHub as a Critical System:** Recognize GitHub (especially GHES) not just as a tool, but as a mission-critical system that underpins development velocity, deployment pipelines, and overall business continuity. Its high availability and robust performance are paramount.
2. **Architectural Mastery:** Deeply understand the internal components of GHES - Git storage, PostgreSQL, Elasticsearch, Redis, and the various application services. Know how they interact, their dependencies, and their failure modes. This knowledge is your map for troubleshooting.
3. **Proactive Observability:**
 - **Monitor Everything:** Leverage GHES's native Prometheus exporter, ``node_exporter``, and centralized log aggregation (``syslog-ng`` to Splunk/ELK/Loki) to gain full visibility into system health, application performance, and security events.
 - **Actionable Alerting:** Craft intelligent Prometheus alerting rules based on clear SLOs (e.g., replication lag, 5xx error rates, disk space, runner availability). Focus on high-signal, low-noise alerts that indicate real problems, not just symptoms.
4. **Robust High Availability & Disaster Recovery:**
 - **Implement HA:** Configure GHES in an active/passive HA setup with a load balancer and understand the mechanics of ``ghe-repl-setup``, ``ghe-repl-status``, and ``ghe-repl-promote``.
 - **Layered DR:** Beyond HA, implement geo-replication for regional disaster recovery and establish a comprehensive ``ghe-backup`` strategy to S3-compatible storage.
 - **Test Relentlessly:** Regular failover and DR drills are non-negotiable. They validate your RTO/RPO objectives and refine your incident response capabilities.
5. **Security-First Mindset:**
 - **Defense in Depth:** Implement strict network isolation, least-privilege IAM, comprehensive encryption (at rest and in transit), and a robust patch management process.
 - **Runner Hardening:** Treat self-hosted runners as highly sensitive, ephemeral workloads. Isolate them, use minimal images, and integrate with a secrets manager.

- **Policy Enforcement:** Utilize pre-receive hooks for proactive security and compliance enforcement at the Git push level.

6. Structured Troubleshooting & RCA:

- **Systematic Approach:** When an incident strikes, follow a methodical approach: check core system resources, verify GHES service status, inspect relevant logs, and confirm network connectivity.
- **Command-Line Proficiency:** Master `ghe-` CLI tools (`ghe-repl-status`, `ghe-status`), `ghe-ssh` for OS-level diagnostics, and `ghe-dbconsole` for database introspection.
- **Deep Dive RCA:** After resolution, conduct thorough Root Cause Analysis to understand *why* the incident occurred, identify systemic weaknesses, and implement preventative measures.

7. Automation & Infrastructure as Code:

- **Automate Operations:** From GHES deployment and upgrades to self-hosted runner management and GitHub resource provisioning (Terraform), automate repetitive tasks to reduce errors and improve efficiency.
- **Integrate Seamlessly:** Understand how GitHub integrates with your broader DevOps toolchain (CI/CD, Kubernetes, Ansible) and optimize these connections for reliability and performance.

By diligently applying these principles and continuously deepening your technical understanding, you will not only become proficient in GitHub operations but also a highly valued SRE capable of ensuring the stability and performance of critical development infrastructure in any enterprise environment. This journey is continuous; stay curious, keep learning, and always challenge the status quo to build more resilient systems.

Q41. Troubleshooting GitHub Actions Runner Disk Space Exhaustion

Detailed Answer:

Disk space exhaustion on self-hosted GitHub Actions runners (whether deployed as virtual machines or Kubernetes pods) is one of the most common causes of pipeline failures in high-throughput CI/CD environments. This occurs due to uncleaned Docker layers, untracked build artifacts, deep `git clone` operations, and package manager caches (such as `.npm`, `.m2`, or pip caches) accumulating over successive runs.

To diagnose and permanently solve this issue, an SRE must design a multi-layered self-healing architecture:

1. **Real-time Monitoring & Alerting:** Instrument runners using the Prometheus `node\_exporter` to track disk space availability.
2. **Automated Pruning:** Deploy aggressive cleanup policies using ephemeral runner patterns or cron-based systemd services.
3. **Storage Allocation Policies:** Separate the OS partition from the runner workspace partition to prevent system-wide lockups.

When a disk exhaustion event occurs, the SRE diagnostic runbook should immediately trace the largest disk consumers using:

```
# Find the top 10 largest directories in the runner workspace
du -ahx /actions-runner/_work/ | sort -rh | head -n 10
```

```
# Inspect Docker disk utilization (if using Docker-in-Docker or containerized steps)
docker system df
```

To prevent this, runners should be configured as ephemeral (using the `--ephemeral` flag during runner registration). Ephemeral runners process exactly one job and are then automatically torn down, ensuring that the local workspace is completely destroyed. If virtual machines are reused, a systemd timer must run a pruning script during idle periods.

Production Scenario / Practical Example:

An SRE team experiences recurrent build failures on their AWS EC2-based self-hosted runner pool with the error ``No space left on device``.

To resolve this, the team implements a Prometheus alert rule and pairs it with an automated self-healing systemd service on the runners.

1. Prometheus Alerting Rule (`/etc/prometheus/rules/runners.rules.yml`)

This rule triggers when disk space on any runner falls below 15% and is projected to empty within 4 hours.

```
groups:
  - name: github_runner_alerts
    rules:
      - alert: RunnerDiskSpaceCriticallyLow
        expr: |
          (node_filesystem_free_bytes{mountpoint="/actions-runner"} * 100) /
          node_filesystem_size_bytes{mountpoint="/actions-runner"} < 15
          and
          predict_linear(node_filesystem_free_bytes{mountpoint="/actions-runner"}[1h], 14400)
          < 0
        for: 5m
        labels:
          severity: critical
          tier: ci-cd
        annotations:
          summary: "Runner {{ $labels.instance }} is running out of disk space"
          description: "Disk space on /actions-runner is below 15% (Current: {{ printf
          \"%.2f\" $value }}%)."
```

2. Self-Healing Script (`/usr/local/bin/runner-cleanup.sh`)

This script is triggered by a cron job or a systemd timer to aggressively prune unused Docker and build cache assets without interrupting active jobs.

```
#!/usr/bin/env bash
set -euo pipefail

THRESHOLD_PERCENT=80
CURRENT_USAGE=$(df -h /actions-runner | awk 'NR==2 {print $5}' | sed 's/%//')
```

```
echo "Current disk usage on /actions-runner: ${CURRENT_USAGE}%"

if [ "${CURRENT_USAGE}" -gt "${THRESHOLD_PERCENT}" ]; then
    echo "Disk usage exceeds threshold of ${THRESHOLD_PERCENT}%. Initiating cleanup..."

    # 1. Prune unused Docker containers, networks, and images older than 24h
    if systemctl is-active --quiet docker; then
        echo "Pruning Docker resources..."
        docker system prune -a --volumes --force --filter "until=24h"
    fi

    # 2. Clean package manager caches
    echo "Cleaning package caches..."
    rm -rf /home/runner/.npm/_cacache/*
    rm -rf /home/runner/.cache/pip/*
    rm -rf /home/runner/.m2/repository/*

    # 3. Clean up orphaned workspace directories (older than 3 days)
    find /actions-runner/_work/_tool/ -mindepth 1 -mtime +3 -exec rm -rf {} + || true

    echo "Cleanup complete. New disk usage: $(df -h /actions-runner | awk 'NR==2 {print $5}')"
else
    echo "Disk usage is within safe parameters."
fi
```

Q42. Diagnosing and Mitigating GitHub API Rate Limiting

Detailed Answer:

At enterprise scale, thousands of CI/CD workflows, ChatOps bots, and developer tools query the GitHub API simultaneously. For GitHub Enterprise Cloud (GHEC) and GitHub Enterprise Server (GHES), API rate limits are strictly enforced:

- **GitHub App Installations**: Up to 12,500 requests per hour (scales with organization size).
- **Personal Access Tokens (PATs)**: 5,000 requests per hour per user.
- **Unauthenticated requests**: 60 requests per hour.

When a rate limit is breached, GitHub returns an HTTP status code `403 Forbidden` with the header `X-RateLimit-Remaining: 0`. SREs must diagnose this by identifying the offending token, IP address, or GitHub App, and then implement a caching proxy and token rotation architecture.

Diagnostic Workflow:

1. Analyze API Headers: Capture the response headers of the failing requests:

- `X-RateLimit-Limit`: The maximum number of requests permitted per hour.
- `X-RateLimit-Remaining`: The number of requests remaining in the current rate limit window.
- `X-RateLimit-Reset`: The Unix epoch time when the current rate limit window resets.

2. Audit GitHub App / Token Usage: Query the GitHub Enterprise Audit Log API or GHES syslog endpoints to aggregate API usage by `actor` or `token\_id`.

3. Trace Callers: Implement distributed tracing (W3C Trace Context) in internal CI/CD tooling to track down recursive API calls (e.g., a script calling `GET /repos/{owner}/{repo}/pulls` inside a loop instead of using pagination or GraphQL).

Mitigation Strategies:

- **GraphQL Migration**: Convert REST API calls to GraphQL to fetch nested payloads in a single request, drastically reducing the total request count.
- **Caching Proxy**: Deploy a caching proxy (such as Squid, Varnish, or an Envoy-based proxy) to cache static read requests (e.g., fetching repository metadata or team memberships).
- **Token Pool / Rotation**: Implement a dynamic vault-backed token broker that rotates GitHub App installations and PATs.

Production Scenario / Practical Example:

An enterprise platform suffers from intermittent pipeline failures. The runner logs reveal:

```
`HTTP/2 403 Forbidden - API rate limit exceeded for user ID xxxxx.`
```

The SRE deploys an Envoy Proxy acting as a caching layer for all outgoing GitHub REST API requests, caching `/orgs` and `/repos` read paths for 5 minutes, and configures a Prometheus metric to alert on impending rate-limit exhaustion.

1. Envoy Proxy Caching Configuration (`/etc/envoy/envoy.yaml`)

```
static_resources:
  listeners:
  - name: github_api_proxy
    address:
      socket_address: { address: 0.0.0.0, port_value: 8080 }
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type":
            type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.HttpConnectionManager
          stat_prefix: github_proxy
          route_config:
            name: github_route
            virtual_hosts:
            - name: github_api
```

```

    domains: ["*"]
    routes:
      - match: { prefix: "/api/v3" } # For GHES, or "api.github.com" for GHEC
        route:
          cluster: github_backend
          host_rewrite_literal: api.github.com
          timeout: 15s
    http_filters:
      - name: envoy.filters.http.cache
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.http.cache.v3.CacheConfig
        typed_config:
          "@type":
type.googleapis.com/envoy.extensions.filters.http.cache.v3.SimpleHttpCacheConfig
      - name: envoy.filters.http.router
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.http.router.v3.Router
    clusters:
      - name: github_backend
        connect_timeout: 5s
        type: LOGICAL_DNS
        dns_lookup_family: V4_ONLY
        lb_policy: ROUND_ROBIN
        transport_socket:
          name: envoy.transport_sockets.tls
          typed_config:
            "@type":
type.googleapis.com/envoy.extensions.transport_sockets.tls.v3.UpstreamTlsContext
          sni: api.github.com
        load_assignment:
          cluster_name: github_backend
          endpoints:
            - lb_endpoints:
                - endpoint:
                    address:
                      socket_address: { address: api.github.com, port_value: 443 }

```

2. Prometheus Metric Alert for Impending Limit Exhaustion

This alert notifies SREs via Alertmanager when the remaining API quota drops below 20% of the limit.

```

- alert: GitHubApiRateLimitCritical
  expr: |
    github_api_rate_limit_remaining{integration="jenkins-master"}
    /
    github_api_rate_limit_limit{integration="jenkins-master"} * 100 < 20
  for: 10m
  labels:

```

```
severity: warning
annotations:
  summary: "GitHub API Quota Exhaustion Risk"
  description: "The integration {{ $labels.integration }} has used over 80% of its API
quota. Remaining requests: {{ $value }}."
```

Q43. GitHub Enterprise Server (GHES) High Availability Split-Brain Recovery

Detailed Answer:

GitHub Enterprise Server (GHES) in High Availability (HA) configuration relies on an active-passive topology. Replication from the primary node to the replica node is managed via a suite of internal services:

- **MySQL**: Replicated asynchronously via MySQL replication.
- **Elasticsearch**: Replicated via native cluster indexing.
- **Git Data (Spokes)**: Replicated via GitHub's proprietary Git storage routing service (Spokes).
- **Redis/Memcached**: Synchronized or cold-started.

A Split-Brain scenario occurs when network partition isolates the primary and replica nodes, causing both nodes to believe they are the authoritative primary. If client traffic is routed to both nodes simultaneously, write divergence occurs, leading to data corruption and divergent Git histories.

Diagnostic Steps:

1. Log into the replica node via SSH and check the replication status:

```
ghe-repl-status
```

2. Inspect the output for replication lag or errors. If both nodes show active web interfaces and accept writes, split-brain is confirmed.

3. Check the Consul cluster state to verify which node holds the leader locks:

```
ghe-cluster-status
```

Recovery Runbook (Force Demotion & Resync):

To recover from a split-brain, you must choose one node as the authoritative source of truth (typically the original primary), discard divergent writes on the other node, demote it back to a replica, and re-initiate replication.

Production Scenario / Practical Example:

Following a network partition in a multi-region deployment, both `ghes-primary.corp` and `ghes-replica.corp` are accepting Git writes. The SRE team executes the following disaster recovery runbook to force-align the cluster.

```
# =====
# STEP 1: ISOLATE THE REPLICA NODE
```

```
# =====
# Log into the replica node (ghes-replica) and stop all front-end services
# to prevent any further writes from clients.
ssh -p 122 admin@ghes-replica.corp -- "ghe-config-apply --stop"

# Stop replication services on the replica
ssh -p 122 admin@ghes-replica.corp -- "ghe-repl-stop"

# =====
# STEP 2: VERIFY PRIMARY INTEGRITY
# =====
# Ensure the primary node is fully functional and accepting traffic
ssh -p 122 admin@ghes-primary.corp -- "ghe-system-info"
ssh -p 122 admin@ghes-primary.corp -- "ghe-repl-status"

# =====
# STEP 3: RE-INITIALIZE REPLICATION ON REPLICA
# =====
# On the replica, force a synchronization of the configuration database (MySQL)
# and overwrite local divergent data with primary data.
ssh -p 122 admin@ghes-replica.corp << 'EOF'
    # Force setup of replication pointing to the primary
    ghe-repl-setup ghes-primary.corp --force

    # Start replication processes
    ghe-repl-start
EOF

# =====
# STEP 4: MONITOR REPLICATION PROGRESS
# =====
# Run an active loop to monitor status until all services return "OK"
echo "Monitoring replication sync..."
for i in {1..30}; do
    STATUS=$(ssh -p 122 admin@ghes-replica.corp "ghe-repl-status --json" | jq -r '.status')
    if [ "$STATUS" = "healthy" ]; then
        echo "Replication is fully synchronized and healthy."
        break
    fi
    echo "Syncing... attempt $i/30"
    sleep 10
done
```

Prometheus Alert to Detect Replication Desync (/etc/prometheus/rules/ghes.rules.yml):

```
- alert: GHESReplicationLagDetected
  expr: |
    github_enterprise_replication_lag_seconds{service="mysql"} > 300
```

```
or
github_enterprise_replication_status{service="spokes"} == 0
for: 5m
labels:
  severity: critical
annotations:
  summary: "GHES Replica Node Out of Sync"
  description: "The replica node has replication lag exceeding 5 minutes, or the Git Spokes
service replication has failed."
```

Q44. Debugging Silent Webhook Delivery Failures

Detailed Answer:

GitHub webhooks are critical for triggering automated CI/CD pipelines (e.g., ArgoCD syncs, Jenkins builds, Tekton triggers). When webhooks fail silently, developers experience "stuck" pipelines where commits are pushed, but no workflows start.

Architectural Vulnerabilities:

1. Network Path Failures: Corporate firewalls, WAFs (Cloudflare, AWS WAF), or Ingress Controllers rejecting payloads due to size or missing headers.
2. TLS Handshake Failures: GitHub endpoints failing to negotiate TLS with internal endpoints using self-signed or expired certificates.
3. Target Exhaustion: The target webhook receiver (e.g., an internal Jenkins controller) suffers from thread pool exhaustion, dropping connections or returning HTTP `503 Service Unavailable`.

SRE Troubleshooting Pipeline:

1. Query GitHub Webhook Delivery API: Use the GitHub API to fetch the delivery status of a repository's webhooks:

```
curl -H "Authorization: token $GITHUB_TOKEN" \
  -H "Accept: application/vnd.github.v3+json" \
  https://api.github.com/repos/{owner}/{repo}/hooks/{hook_id}/deliveries
```

Inspect the `status\_code`, `duration`, and `response` fields.

2. Verify Request Signatures: Ensure the receiver is validating the `X-Hub-Signature-256` header correctly. A common bug is mismatching cryptographic keys due to trailing newlines in secrets.
3. Decouple with an Asynchronous Queue: To prevent target exhaustion, route incoming webhooks to an API Gateway backed by an Amazon SQS queue or RabbitMQ broker, which decouples ingestion from ingestion processing.

Production Scenario / Practical Example:

An enterprise SRE team notices that webhook deliveries from GitHub Enterprise Cloud to their internal Kubernetes ingress are intermittently failing with `504 Gateway Timeout` during peak hours.

To solve this, they deploy a lightweight, highly available Go-based webhook receiver that puts payloads directly into a Redis Queue for consumption by CI/CD tools, eliminating direct-to-app coupling.

```
[GitHub Cloud] ??(HTTPS Webhook)?> [Kubernetes Ingress]
                                ?
                                ?
                        [Go Webhook Receiver]
                                ?
                        (LPUSH Payload)
                                ?
                        [Redis Queue]
                                ?
                        (RPOP / Process)
                                ?
                        [Internal SRE Consumer]
```

1. Go Webhook Receiver Snippet (Ingestion Layer)

This handler validates signatures, writes payloads directly to Redis, and returns an immediate HTTP `202 Accepted` to GitHub within 100 milliseconds.

```
package main

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/hex"
    "io"
    "net/http"
    "github.com/go-redis/redis/v8"
)

var ctx = context.Background()
var rdb = redis.NewClient(&redis.Options{Addr: "redis-master:6379"})
const WebhookSecret = "super-secret-key"

func WebhookHandler(w http.ResponseWriter, r *http.Request) {
    signature := r.Header.Get("X-Hub-Signature-256")
    payload, _ := io.ReadAll(r.Body)

    // Validate Signature
    mac := hmac.New(sha256.New, []byte(WebhookSecret))
    mac.Write(payload)
    expectedMAC := "sha256=" + hex.EncodeToString(mac.Sum(nil))
```

```

if !hmac.Equal([]byte(signature), []byte(expectedMAC)) {
    http.Error(w, "Invalid Signature", http.StatusUnauthorized)
    return
}

// Push raw payload to Redis list for asynchronous processing
err := rdb.LPush(ctx, "github_webhooks", payload).Err()
if err != nil {
    http.Error(w, "Queue Failure", http.StatusInternalServerError)
    return
}

w.WriteHeader(http.StatusAccepted)
w.Write([]byte(`{"status":"queued"}`))
}

```

2. Prometheus Alert for Webhook Delivery Failures (Target Side)

```

- alert: WebhookReceiverHighErrorRate
  expr: |
    sum(rate(http_requests_total{handler="webhook", status=~"5.*"}[5m]))
    /
    sum(rate(http_requests_total{handler="webhook"}[5m])) * 100 > 5
  for: 2m
  labels:
    severity: page
  annotations:
    summary: "Webhook receiver error rate exceeds 5%"
    description: "The ingress webhook receiver is failing to ingest hooks with HTTP 5xx
errors. Current failure rate: {{ printf \"%.2f\" $value }}%."

```

Q45. Troubleshooting Git Large File Storage (LFS) Quota and Corruption Issues

Detailed Answer:

Git LFS replaces large files (such as audio, video, datasets, and binaries) with tiny text pointers inside Git, while storing the actual file content on an external storage server (e.g., AWS S3, Azure Blob, or local SAN storage configured in GHES).

SREs frequently encounter two failure modes with Git LFS:

1. LFS Pointer Corruption / Missing Objects: A developer pushes a commit containing an LFS pointer, but the actual asset transfer fails or is interrupted. Subsequent clones fail with:

`Error: LFS attribute not found` or `Error: download failed: Object does not exist on the LFS server`.

2. Storage Exhaustion: LFS storage consumption grows exponentially because Git LFS keeps history of every

version of a binary file.

Diagnostic and Auditing Workflow:

- ****Locate Corrupt Pointers****: Use the ``git lfs fsck`` command on a local clone to verify the integrity of LFS objects:

```
git lfs fsck
```

- ****Server-side Auditing (GHES)****: SSH into the GHES instance and query the administrative utilities to locate orphaned LFS objects (objects in storage that have no corresponding pointer in any Git commit):

```
ghe-spokes-dht-query -c "lfs-check"
```

- ****LFS Object Purging****: To recover storage, use tools like ``git-filter-repo`` to rewrite history and completely strip unneeded legacy LFS files, then trigger garbage collection on the GHES appliance.

Production Scenario / Practical Example:

An SRE is paged because developers cannot clone a critical monorepo. The clone fails with:

```
`[404] Object not found on LFS server (OID: a3b8f9e1...)`.
```

The SRE executes a recovery runbook to identify the broken commit, bypass the missing object to restore clone functionality, and clean up orphaned objects on the server.

SRE Recovery Script:

```
# =====
# STEP 1: IDENTIFY THE COMMITS REFERENCING THE CORRUPT LFS OID
# =====
CORRUPT_OID="a3b8f9e1fc22d4f55a1d072b226e3c59a3c75196aa0e1e69b012354a8e87412b"

echo "Searching for commits referencing OID: ${CORRUPT_OID}"
git log --all --geom-orders --find-object="${CORRUPT_OID}"

# If the file is lost forever from the developer's workstation, we must force-purge
# the pointer from Git history to allow others to clone.
# We use git-filter-repo to remove the path containing the broken file.

# =====
# STEP 2: REWRITE HISTORY TO EXCLUDE CORRUPT PATHS
# =====
pip install git-filter-repo
git filter-repo --path "path/to/corrupt-large-file.bin" --invert-paths --force

# Force push the repaired history back to GitHub (requires administrative privileges)
git push origin --all --force --tags

# =====
# STEP 3: SERVER-SIDE GARBAGE COLLECTION (GHES)
```

```
# =====  
# Log into the GHES instance to reclaim storage immediately  
ssh -p 122 admin@ghes-instance.corp << EOF  
# Start garbage collection for the repository  
ghe-repo-gc --repository "org-name/monorepo" --force  
EOF
```

Prometheus Alerting Rule for LFS Storage Exhaustion (GHES Appliance)

```
- alert: GHESLfsStorageNearLimit  
  expr: |  
    github_enterprise_storage_used_bytes{storage_type="lfs"}  
    /  
    github_enterprise_storage_total_bytes{storage_type="lfs"} * 100 > 90  
  for: 1h  
  labels:  
    severity: warning  
  annotations:  
    summary: "GHES Git LFS storage exceeds 90%"  
    description: "Git LFS storage volume is at {{ printf \"%.2f\" $value }}% capacity. Please  
scale the underlying EBS volume or run LFS retention policies."
```

Q46. Mitigating Actions Runner Controller (ARC) Starvation during Peak Windows

Detailed Answer:

GitHub Actions Runner Controller (ARC) is a Kubernetes operator that orchestrates self-hosted runners as pods inside a Kubernetes cluster. During peak deployment windows (e.g., mid-day on Tuesdays), the volume of queued jobs spikes dramatically.

If ARC is misconfigured, runner starvation occurs: jobs sit in a "Queued" state for 20+ minutes, blocking critical hotfixes.

Bottlenecks in ARC Autoscaling:

1. GitHub API Rate Limits: ARC queries GitHub APIs (`GET /orgs/{org}/actions/runners`) to calculate queue length. High polling frequencies quickly exhaust API limits.
2. Kubernetes Scheduler Lag: Pod creation, image pulling (especially heavy build images containing Android SDKs or heavy compilers), and node provisioning (via cluster-autoscaler or Karpenter) add minutes of latency.
3. Webhook-driven vs. Pull-driven Scaling: Pull-driven scaling is slow and rate-limited. Webhook-driven autoscaling is instantaneous but vulnerable to dropped webhook events.

SRE Architectural Solutions:

- ****Implement KEDA (Kubernetes Event-driven Autoscaling)****: Instead of the default ARC metrics API, use KEDA to scale runner pods. KEDA listens to GitHub Enterprise Webhooks directly or queries metrics from Prometheus.
- ****Image Pre-warming (DaemonSets)****: Build a custom DaemonSet that pulls the runner build images onto all Kubernetes worker nodes nightly, reducing pod startup latency from 3 minutes to 5 seconds.
- ****Karpenter Node Provisioning****: Configure Karpenter to provision spot/on-demand instances instantly using custom node templates optimized for memory-intensive builds.

Production Scenario / Practical Example:

An SRE team experiences massive pipeline delays. They migrate their ARC setup from pull-based scaling to a webhook-triggered KEDA autoscaling system using a `ScaledObject` pointing to the GitHub Actions metrics exporter.

KEDA ScaledObject and RunnerDeployment Configuration (`/deploy/arc-autoscaler.yaml`)

```
apiVersion: actions.summerwind.dev/v1alpha1
kind: RunnerDeployment
metadata:
  name: dynamic-runner-pool
  namespace: arc-systems
spec:
  replicas: 1 # Keep a warm base runner
  template:
    spec:
      repository: corp-org/high-frequency-repo
      image: ghcr.io/actions/runner:latest
      resources:
        requests:
          cpu: "2"
          memory: "4Gi"
        limits:
          cpu: "4"
          memory: "8Gi"
---
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: github-runner-autoscaler
  namespace: arc-systems
spec:
  scaleTargetRef:
    apiVersion: actions.summerwind.dev/v1alpha1
    kind: RunnerDeployment
    name: dynamic-runner-pool
  minReplicaCount: 1
  maxReplicaCount: 100
  cooldownPeriod: 300
```

```
advanced:
  horizontalPodAutoscalerConfig:
    behavior:
      scaleUp:
        stabilizationWindowSeconds: 0
        policies:
          - type: Percent
            value: 100
            periodSeconds: 15
    triggers:
      - type: prometheus
    metadata:
      serverAddress: http://prometheus-k8s.monitoring.svc:9090
      metricName: github_workflow_jobs_queued_total
      # Scale based on the number of queued workflow runs extracted from the GitHub Exporter
      query: sum(github_workflow_jobs_status{status="queued", repo="high-frequency-repo"})
      threshold: '1'
```

Q47. Recovering a Corrupted Git Repository Server-Side on GHES

Detailed Answer:

Git repository corruption on a self-hosted GitHub Enterprise Server (GHES) instance can occur due to underlying disk write errors, hard node lockups, filesystem journal corruption, or buggy custom Git hooks. This manifests to users as failure to clone, push, or view files, returning errors such as:

`error: object xxxxx is corrupted` or `fatal: loose object xxxxx is corrupt`.

Because GHES stores repositories under `/data/user/repositories/` in raw bare Git directories, SREs must use Git plumbing commands directly on the server backend to reconstruct references.

SRE Diagnostics on GHES:

1. SSH to the GHES instance on administrative port `122`.
2. Locate the physical path of the repository. GHES hashes repository paths; use the administrative command line to find the exact path:

```
ghe-repo-path "org-name/corrupt-repo"
```

Output looks like: `/data/user/repositories/0/nw/12/34/56/7890/network.git`

3. Run `git fsck` inside the target directory to identify loose corrupt objects, dangling commits, or broken references.

Recovery Strategy:

- ****If HA is healthy****: Force-overwrite the corrupt repository by replicating data from the healthy passive replica node.
- ****If HA is also corrupted****: Recover missing loose objects from local client developer workstations or from the latest daily `ghe-backup` snapshot.

Production Scenario / Practical Example:

A core repository gets corrupted. Developers get `fatal: bad object HEAD` on every fetch. The SRE SSHes into the GHES node to repair the loose object corruptions.

```
# =====
# STEP 1: SSH AND LOCATE THE REPOSITORY PATH
# =====
ssh -p 122 admin@ghes-primary.corp << 'EOF'
  REPO_PATH=$(ghe-repo-path "platform/core-api")
  echo "Repository path is: ${REPO_PATH}"
  cd "${REPO_PATH}"

  # Run low-level Git checks
  echo "Executing git fsck..."
  git fsck --full --unreachable > /tmp/fsck_results.txt || true
EOF

# =====
# STEP 2: REPAIR LOOSE OBJECT CORRUPTION (RUN ON GHES NODE)
# =====
# If git fsck reports: "error: object file .git/objects/ab/1234... is empty"
# We must find and remove the empty/corrupt object file, then pull it from a replica
# or ask a developer with a fresh copy to push it back.

ssh -p 122 admin@ghes-primary.corp << 'EOF'
  cd $(ghe-repo-path "platform/core-api")

  # Find 0-byte (empty/corrupt) object files and remove them
  find objects/ -type f -empty -delete

  # If we have a High Availability replica, force a Spokes resync from the replica
  # to overwrite the corrupt repository with the healthy replica state.
  echo "Triggering server-side Spokes sync..."
  ghe-spokes-ctl repair "platform/core-api"
EOF

# =====
# STEP 3: MANUALLY RESTORING MISSING REFS FROM REFLOG
# =====
# If Spokes sync is unavailable, we attempt to reconstruct HEAD from the local reflog:
ssh -p 122 admin@ghes-primary.corp << 'EOF'
  cd $(ghe-repo-path "platform/core-api")
```

```
# Read the last valid commit from the reflog
LAST_VALID_COMMIT=$(tail -n 1 logs/refs/heads/main | awk '{print $2}')

if [ ! -z "$LAST_VALID_COMMIT" ]; then
    echo "Pointing HEAD back to last valid commit: ${LAST_VALID_COMMIT}"
    echo "${LAST_VALID_COMMIT}" > refs/heads/main

    # Re-run fsck to verify recovery
    git fsck --non-strict
else
    echo "Reflog empty. Restoring from last backup snapshot..."
fi
EOF
```

Q48. Implementing Real-Time Secret Leakage Detection and Automated Revocation

Detailed Answer:

When developers accidentally commit secrets (AWS Access Keys, GitHub PATs, Slack Webhooks, database credentials) to git repositories, the time-to-exploit by malicious actors scanning public and internal systems is measured in seconds.

Relying on periodic nightly scans is insufficient. SREs must implement an event-driven real-time detection and automated containment pipeline:

1. GitHub Secret Scanning: Enable GitHub's native Secret Scanning (for public repos or via Advanced Security for private repos).
2. Event Webhook: Configure GitHub Organization webhooks to listen specifically for the `secret\_scanning\_alert` event.
3. Automated Serverless Remediation: Route the webhook to an AWS Lambda function or Kubernetes microservice. The service parses the secret type, validates the token via dry-run API queries, and instantly disables/revokes the credential via AWS IAM, HashiCorp Vault, or the target platform API.

Production Scenario / Practical Example:

An SRE designs a zero-trust automation pipeline. When a developer pushes an AWS IAM access key to any repository, the pipeline disables the AWS IAM user within 5 seconds of the push.

```
[Developer Push] ??> [GitHub Repo (Secret Scanning)]
                        ?
                    (Webhook Alert)
                        ?
                    [AWS API Gateway]
                        ?
                    [SRE Lambda Worker]
```

```
??? 1. Parse Alert Metadata
??? 2. Trigger AWS IAM API
??? 3. Send Slack/PagerDuty Alert
```

1. AWS Lambda Python Script (`lambda\_function.py`)

This script processes the `secret\_scanning\_alert` webhook, extracts the leaked AWS Access Key ID, and immediately deactivates it.

```
import json
import boto3
import urllib3

iam_client = boto3.client('iam')
http = urllib3.PoolManager()

SLACK_WEBHOOK_URL = "https://hooks.slack.com/services/T00/B00/X00"

def lambda_handler(event, context):
    body = json.loads(event.get('body', '{}'))
    action = body.get('action')

    # Check if a new secret was detected/created
    if action in ['created', 'reopened']:
        alert = body.get('alert', {})
        secret_type = alert.get('secret_type')
        secret_value = alert.get('secret') # Only provided if authorized

        if secret_type == 'aws_access_key_id':
            access_key_id = alert.get('secret')
            print(f"CRITICAL: Leaked AWS Access Key detected: {access_key_id}")

            # Step 1: Remediate by deactivating the IAM Access Key
            try:
                # Find the owner of the access key
                # Note: In production, you would iterate over IAM users or query CloudTrail
                user_response =
iam_client.get_access_key_last_used(AccessKeyId=access_key_id)
                username = user_response.get('UserName')

                # Deactivate the key immediately
                iam_client.update_access_key(
                    UserName=username,
                    AccessKeyId=access_key_id,
                    Status='Inactive'
                )
                print(f"SUCCESS: Deactivated access key {access_key_id} for user {username}")
                send_slack_alert(username, access_key_id, "DEACTIVATED_AUTOMATICALLY")
```

```
        except Exception as e:
            print(f"FAILED to deactivate key {access_key_id}: {str(e)}")
            send_slack_alert("UNKNOWN", access_key_id, f"FAILED_TO_REVOKE: {str(e)}")

    return {
        'statusCode': 200,
        'body': json.dumps('Event processed')
    }

def send_slack_alert(user, key_id, status):
    payload = {
        "text": f"? *Security Incident*: Leaked AWS Secret Detected!\n"
                f"*User*: `{user}`\n"
                f"*Key ID*: `{key_id}`\n"
                f"*Remediation Status*: `{status}`"
    }
    http.request('POST', SLACK_WEBHOOK_URL, json=payload)
```

2. Prometheus Metric for Tracking Leaked Secrets

```
- alert: HighVolumeSecretLeakage
  expr: |
    sum(increase(github_secret_scanning_alerts_created_total[1h])) > 5
  for: 0m
  labels:
    severity: critical
  annotations:
    summary: "High volume of secrets leaked in the last hour"
    description: "Anomalous developer behavior detected. Over 5 secrets have been pushed in the last hour."
```

Q49. Troubleshooting Network Latency/Packet Loss between GHES and AWS Runners

Detailed Answer:

In hybrid cloud environments, self-hosted GitHub Enterprise Server (GHES) is often hosted on-premises, while GitHub Actions runners are dynamically spun up in AWS VPCs. Intermittent network latency, packet drops, or connection terminations (e.g., `Connection reset by peer` or `fatal: index-pack failed`) will disrupt CI/CD pipelines.

These issues are typically rooted in:

1. MTU Mismatch: The default MTU on AWS EC2 is 9000 (Jumbo Frames), whereas on-premises paths or VPN tunnels (IPsec/Direct Connect) typically enforce an MTU of 1500 or 1420. Large packets are dropped without fragmentation if the DF (Don't Fragment) bit is set.

2. Stateful Firewall Timeout: Firewalls dropping idle TCP connections during long-running compilation steps before the runner can send the final build outputs back to GHES.
3. NAT Gateway Port Exhaustion: A high density of runner pods sharing a single NAT Gateway, exhausting ephemeral ports.

SRE Troubleshooting Pipeline:

- ****Path MTU Discovery Diagnostics****:

```
# Run from the runner terminal to probe the path MTU to the GHES server
ping -M do -s 1472 ghes.corp.internal
# If this fails, decrement size until it succeeds to find the exact path MTU limit
```

- ****eBPF / tcplife Tracking****: Monitor TCP connections on the runner to see where connections are terminated:

```
sudo /usr/share/bcc/tools/tcplife
```

- ****TCP Keepalive Tuning****: Configure runners to send active TCP keepalives to prevent firewall state table timeouts.

Production Scenario / Practical Example:

An SRE team notices that large git clones (over 2GB) consistently fail with `error: RPC failed; curl 56 Recv failure: Connection reset by peer`.

The SRE diagnoses a Path MTU mismatch over the AWS Direct Connect Transit Gateway and implements MTU clamping and TCP keepalive tuning via Daemonset/InitContainers on the Kubernetes runners.

1. Kubernetes Init-Container Configuration to clamp TCP MSS (`/deploy/runner-pod.yaml`)

This Init-Container runs with elevated privileges to clamp the TCP Maximum Segment Size (MSS) on all outgoing connections, ensuring no packet exceeds the on-premises VPN limitations.

```
apiVersion: v1
kind: Pod
metadata:
  name: dynamic-runner-pod
  namespace: arc-systems
spec:
  initContainers:
    - name: network-tuner
      image: alpine:latest
      securityContext:
        capabilities:
          add: ["NET_ADMIN"]
      command:
        - /bin/sh
        - -c
        - |
```

```
# Force MTU to 1420 on the primary ethernet interface to align with VPN tunnel
ip link set dev eth0 mtu 1420

# Configure system-wide TCP keepalive parameters
sysctl -w net.ipv4.tcp_keepalive_time=60
sysctl -w net.ipv4.tcp_keepalive_intvl=10
sysctl -w net.ipv4.tcp_keepalive_probes=6
containers:
- name: runner
  image: ghcr.io/actions/runner:latest
```

2. Prometheus Alert for TCP Retransmissions on Runners

If packet loss is occurring, TCP retransmission rates will spike. This alert warns SREs of underlying network path degradation.

```
- alert: RunnerHighTcpRetransmissionRate
  expr: |
    sum(rate(node_netstat_Tcp_RetransSegs[5m]))
    /
    sum(rate(node_netstat_Tcp_OutSegs[5m])) * 100 > 2
  for: 10m
  labels:
    severity: warning
  annotations:
    summary: "High TCP Retransmission Rate on Runner: {{ $labels.instance }}"
    description: "TCP retransmissions are at {{ printf \"%.2f\" $value }}%. This indicates severe packet loss or MTU mismatches between AWS and GHES."
```

Q50. GHES Upgrade Failure & Rollback Runbook

Detailed Answer:

Upgrading GitHub Enterprise Server (GHES) is a critical operation. While minor hotpatches are low-risk, major upgrades (e.g., 3.8 to 3.10) execute extensive database schema migrations (MySQL, Elasticsearch reindexing, Nomad state updates).

If a migration fails midway due to locked tables, schema deadlocks, or disk exhaustion, the appliance may enter an unbootable state.

An SRE must be prepared to execute a zero-data-loss rollback. This requires a robust disaster recovery workflow:

1. Pre-upgrade Backup: Execute a full snapshot using `ghe-backup` on the designated backup host immediately before beginning the upgrade.
2. Pre-flight Checks: Verify free disk space (at least 20% on `/` and `/data`), and run `ghe-upgrade --dry-run` to

validate package signatures.

3. Upgrade Logs Monitoring: During execution, tail the upgrade log files:

```
tail -f /data/user/common/ghes-upgrade.log
```

4. Rollback Strategy: If the upgrade fails and cannot be quickly patched, the SRE must restore the virtual machine's storage volumes to their pre-upgrade snapshots.

Production Scenario / Practical Example:

During an upgrade from GHES 3.8 to 3.10, the migration script fails at step 87 with a DB deadlock error: `Migration failed: Alter table schema ... Timeout waiting for table lock`. The appliance web interface returns a `503 Service Unavailable` error.

The SRE executes the emergency rollback runbook to restore the cluster to operational status within the maintenance window.

Emergency Rollback Runbook:

```
# =====
# STEP 1: CAPTURE CRITICAL DIAGNOSTIC LOGS FOR ANALYSIS
# =====
# Before rolling back, copy the migration logs to an external storage bucket
ssh -p 122 admin@ghes-primary.corp "cat /data/user/common/ghes-upgrade.log" >
./ghes-failed-upgrade.log

# =====
# STEP 2: POWER DOWN THE FAILED APPLIANCES (AWS EC2 EXAMPLE)
# =====
echo "Stopping primary and replica GHES EC2 instances..."
aws ec2 stop-instances --instance-ids i-0primary12345 i-0replica67890
aws ec2 wait instance-stopped --instance-ids i-0primary12345 i-0replica67890

# =====
# STEP 3: RESTORE ROOT AND DATA VOLUMES TO PRE-UPGRADE SNAPSHOTS
# =====
# Get the Volume IDs of the current corrupted volumes
PRIMARY_ROOT_VOL=$(aws ec2 describe-instances --instance-ids i-0primary12345 --query
"Reservations[0].Instances[0].BlockDeviceMappings[?DeviceName=='/dev/sda1'].Ebs.VolumeId"
--output text)
PRIMARY_DATA_VOL=$(aws ec2 describe-instances --instance-ids i-0primary12345 --query
"Reservations[0].Instances[0].BlockDeviceMappings[?DeviceName=='/dev/sdb'].Ebs.VolumeId"
--output text)

# Detach the corrupted volumes
aws ec2 detach-volume --volume-id ${PRIMARY_ROOT_VOL}
aws ec2 detach-volume --volume-id ${PRIMARY_DATA_VOL}
```

```
# Create new volumes from the pre-upgrade snapshots (e.g., snap-0root111, snap-0data222)
echo "Recreating volumes from pre-upgrade snapshots..."
NEW_ROOT_VOL=$(aws ec2 create-volume --snapshot-id snap-0root111 --availability-zone
us-east-1a --query "VolumeId" --output text)
NEW_DATA_VOL=$(aws ec2 create-volume --snapshot-id snap-0data222 --availability-zone
us-east-1a --query "VolumeId" --output text)

# Wait for volumes to become available
aws ec2 wait volume-available --volume-ids ${NEW_ROOT_VOL} ${NEW_DATA_VOL}

# Attach the restored volumes back to the Primary instance
aws ec2 attach-volume --instance-id i-0primary12345 --volume-id ${NEW_ROOT_VOL} --device
/dev/sda1
aws ec2 attach-volume --instance-id i-0primary12345 --volume-id ${NEW_DATA_VOL} --device
/dev/sdb

# =====
# STEP 4: BOOT APPLIANCE AND VERIFY DATA INTEGRITY
# =====
aws ec2 start-instances --instance-ids i-0primary12345
aws ec2 wait instance-running --instance-ids i-0primary12345

echo "Waiting for GHES services to initialize..."
until curl -s -k -o /dev/null -w "%{http_code}" https://ghes-primary.corp/status | grep
"200"; do
    echo "Checking health status... still initializing..."
    sleep 15
done

# Run final sanity checks on the restored instance
ssh -p 122 admin@ghes-primary.corp << 'EOF'
    ghe-system-info
    ghe-repl-status
EOF
echo "Rollback successfully completed. GHES is operational on the pre-upgrade version."
```